M256 Unit 7
UNDERGRADUATE COMPUTING

Software development with Java

Design decisions

Unit 7

This publication forms part of an Open University course M256 *Software development with Java.* Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom:
tel. +44 (0)845 300 60 90, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at http://www.open.ac.uk where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit http://www.ouw.co.uk, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

The Open University
Walton Hall
Milton Keynes
MK7 6AA
First published 2007.

Edited and designed by The Open University.

Typeset by The Open University.

Printed and bound in Malta by Gutenberg Press.

ISBN 978 0 7492 1568 2

1.1

CONTENTS

M256 COURSE TEAM

Affiliated to The Open University unless otherwise stated.

Sarah Mattingly, Course Chair and Author

Lindsey Court, Author

Marion Edwards, Software Developer

Rob Griffiths, Critical Reader

Benedict Heal, Critical Reader

Simon Holland, Author

Barbara Poniatowska, Course Manager

Barbara Segal, Author

Rita Tingle, Author

Richard Walker, Author and Critical Reader

Robin Walker, Author and Critical Reader

Ray Weedon, Academic Editor

Ray Welland, External Assessor, University of Glasgow

Julia White, Course Manager

Ian Blackham, Editor

Phillip Howe, Compositor

Callum Lester, Software Developer

Andy Seddon, Media Project Manager

Andrew Whitehead, Graphic Artist

Thanks are due to the Desktop Publishing Unit, Faculty of Mathematics and Computing.

1 Introduction

In this unit we continue the design process for the Hospital System that we began in *Unit 6*. In that unit, we created a single design for each use case. A main theme of this unit is the consideration of *different* designs. Some use cases are very simple, and there are no significant different designs to consider; for others, there are many possible variations.

1.1 Different designs

You will be introduced to some important factors to consider when comparing different designs. However, you are not expected, in this introductory course, to be able to decide which the 'best' design is. Indeed, this is something which even professionals often disagree on. An appreciation that different designs are possible and an understanding of some of the factors that can be taken into account when comparing them is all that is expected of you in M256.

In this unit we also consider:

- how to deal with alternative scenarios and situations that could lead to invariants being violated;
- how to deal with derived attributes and associations in designs;
- what a user interface will require of the core system so that it can supply appropriate objects as arguments to coordinating messages and retrieve information required for display to the user.

Bear in mind throughout this unit that designs (expressed in dynamic models) and their consequences for the structure of a system are recorded in the emerging implementation model for the system – the basis upon which the system code will be written. For the Hospital System, this unit will move the implementation model considerably further forward, as shown in the appendix to this unit, but the model will not be completed until the end of *Unit 9*. As progress is made towards the implementation of the system, designs become more specific to the target language for the system. In our case, that is Java, and you will notice Java-specific elements in the designs in this unit. However, the overarching design ideas are still generally applicable to different target languages.

The complete implementation model for the Hospital System, which consists of the structural and dynamic models, is included in Section 5 of the *Handbook*.

1.2 The aims of this unit

In this unit we aim to:

- investigate a different design for one of the Hospital System use cases (Section 2);
- explain some factors that are relevant when comparing different designs (Section 2);
- investigate alternative and invalid scenarios for some use cases in the Hospital System (Section 3);
- consider how to ensure that invariants are preserved (Section 3);
- discuss how derived associations and attributes may be dealt with during design (Section 4);

- explain what additional methods are needed to provide the user interface with the objects and attribute values it needs (Section 5).

1.3 Studying this unit

This unit is shorter than *Unit 6*, and it introduces few new concepts. You may find that it takes less than the time indicated on the *Study Calendar* to complete.

This unit contains a number of SAQs and paper-based exercises which are designed to reinforce your understanding of the concepts presented. These form an important part of the teaching strategy and you should work through them as they arise, and read the solutions provided before moving on. There is no computer-based work in this unit, but some of the exercises are quite substantial.

We expect that each section should require approximately the same study time.

This unit refers to the *Handbook*, which you should have to hand as you study. The implementation model in Section 5 of the *Handbook* incorporates all design decisions taken in the development of the Hospital System. In the spirit of iterative development, some of the designs in this current unit (whose consequences for the structural model are recorded in this unit's appendix) are refined later in the development process, and therefore do not exactly match the final versions recorded in the implementation model.

2 Comparing designs

In *Unit 6* we began the design process for some of the Hospital System use cases. For each, we created *one* design for the core system's behaviour. In a real software development project, a significant proportion of a developer's time may be spent in comparing *different* designs for the same task. In most cases there is no single best design, but some designs may be preferable to others because they are likely to lead to better-quality software, that is, software with such desirable qualities as maintainability and reusability.

Since the designs for different use cases are often dependent, in evaluating different designs for a use case it is necessary to consider the impact of each design on those for other use cases. It is only feasible to decide which design to follow for a particular use case when those for related use cases have been considered. The business of evaluating the impact of different designs can be complex, demanding skills that need to be honed with practice. Our aims here are more modest: you should gain an appreciation of what can be involved in evaluating designs, and an ability to analyse the consequences of adopting a different design in tightly prescribed circumstances.

In this section, we revisit one of the Hospital System's use cases, and examine an alternative design for it. Using this example, you will be introduced to some basic guiding principles used by developers in evaluating designs.

2.1 A different design

Here is the description of the *List Team's Patients* use case (labelled F in the requirements document) from the Hospital System.

> The administrator identifies the team. For each patient cared for by the team the system displays:
>
> - the patient's name;
> - the name of the ward that the patient is on.

SAQ 1

What object does the user interface pass to the coordinating object to initiate the use case? What does the core system return to the user interface?

ANSWER.....

The user interface passes the relevant `Team` object to the coordinating object. The coordinating object returns a collection of pairs of `Patient` and `Ward` objects.

The coordinating method, invoked when the coordinating object receives a coordinating message from the user interface, is in this case `getPatientsAndWards(aTeam)`. Here is the specification we set out in *Unit 6.*

```
Map<Patient, Ward> getPatientsAndWards(Team aTeam)
```

> Post-condition: returns a map of pairs of `Patient` and `Ward` objects, such that for each `Patient` object, `aPatient`, linked to `aTeam`, the map contains the key–value pair (`aPatient`, `aWard`), where `aWard` is linked to `aPatient`.

And here is the dynamic model that we created for this method. This comprises a walk-through and sequence diagram for a typical scenario – a typical application of the use case involving particular example objects. We will refer to this as Design 1 for this use case.

Design 1

Figure 1 shows the objects and links for the scenario.

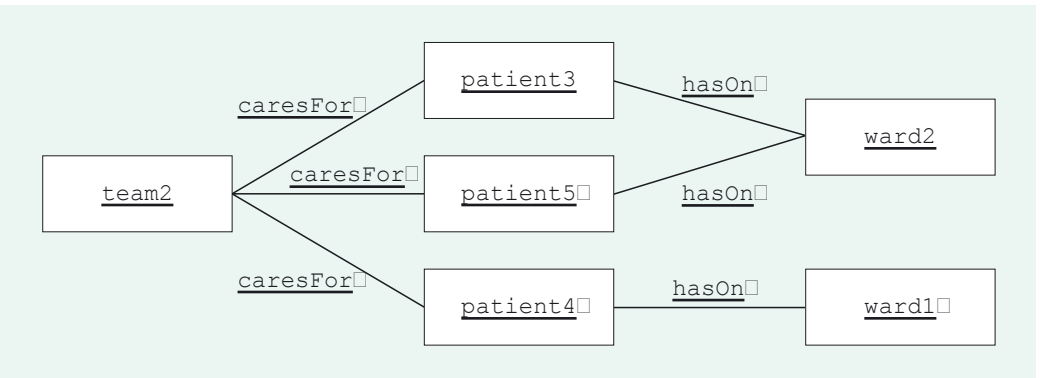

Figure 1 Object diagram for *List Team's Patients* typical scenario

This dynamic model is also set out in Section 5 of the *Handbook*. You may wish to have this to hand as you study this section.

Walk-through:

Given: the `Team` object `team2`.

1 Access the collection of `Patient` objects linked to `team2`, which in this case is {`patient3`, `patient4`, `patient5`}.

2 Create a new empty collection, `results`.

For each `Patient` object, `aPatient`, accessed in step 1:

3 access `aWard`, the `Ward` object linked to `aPatient`

4 add (`aPatient`, `aWard`) to `results`.

5 Return `results`.

Figure 2 shows the sequence diagram.

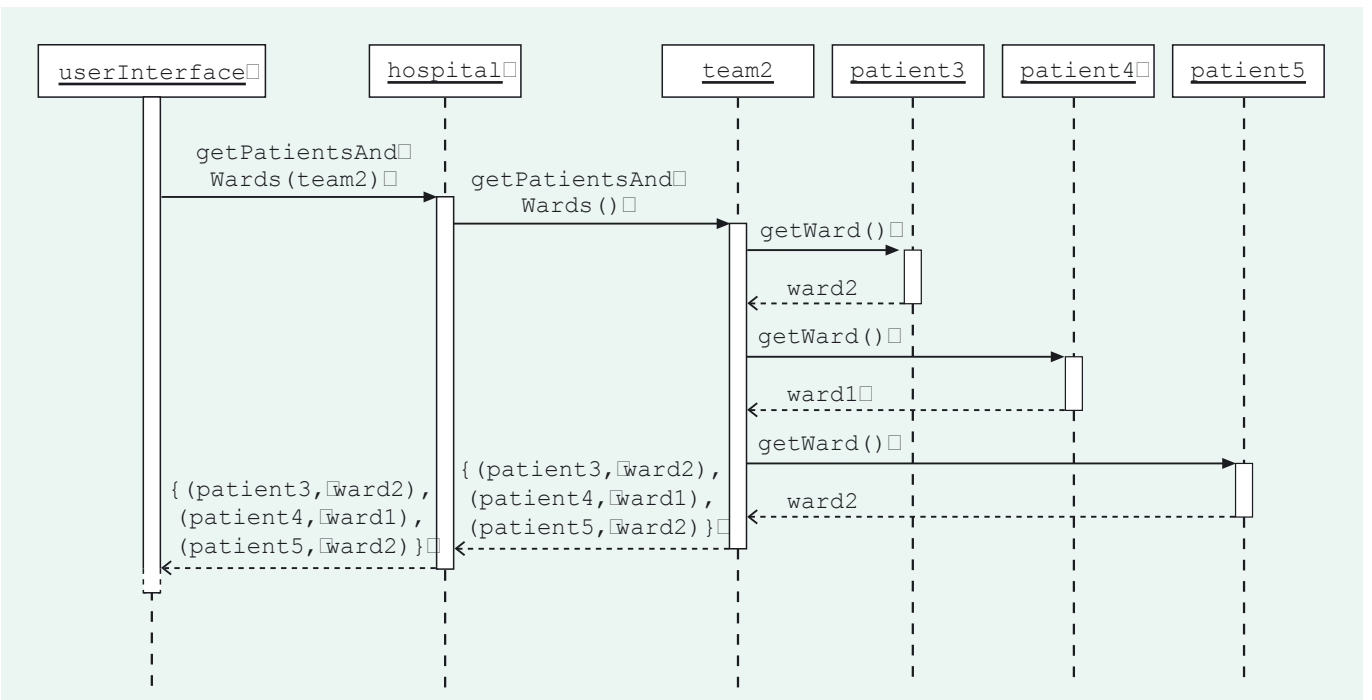

Figure 2 `getPatientsAndWards(aTeam)`: Design 1's sequence diagram

In this design, `team2` is responsible for asking each of its `Patient` objects which ward the patient is on, and it does this by sending `getWard()` messages. However, there are other possibilities for this interrogation of the `Patient` objects.

Design 2

Another possibility is that the coordinating object, `hospital`, could communicate directly with the `Patient` objects: `hospital` could first ask `team2` for its `Patient` objects, and then send a `getWard()` message to each of the `Patient` objects.

We will refer to this alternative as Design 2. The walk-through for Design 2 is the same as that for Design 1, but the steps are achieved as we have just described. A corresponding sequence diagram is shown in Figure 3.

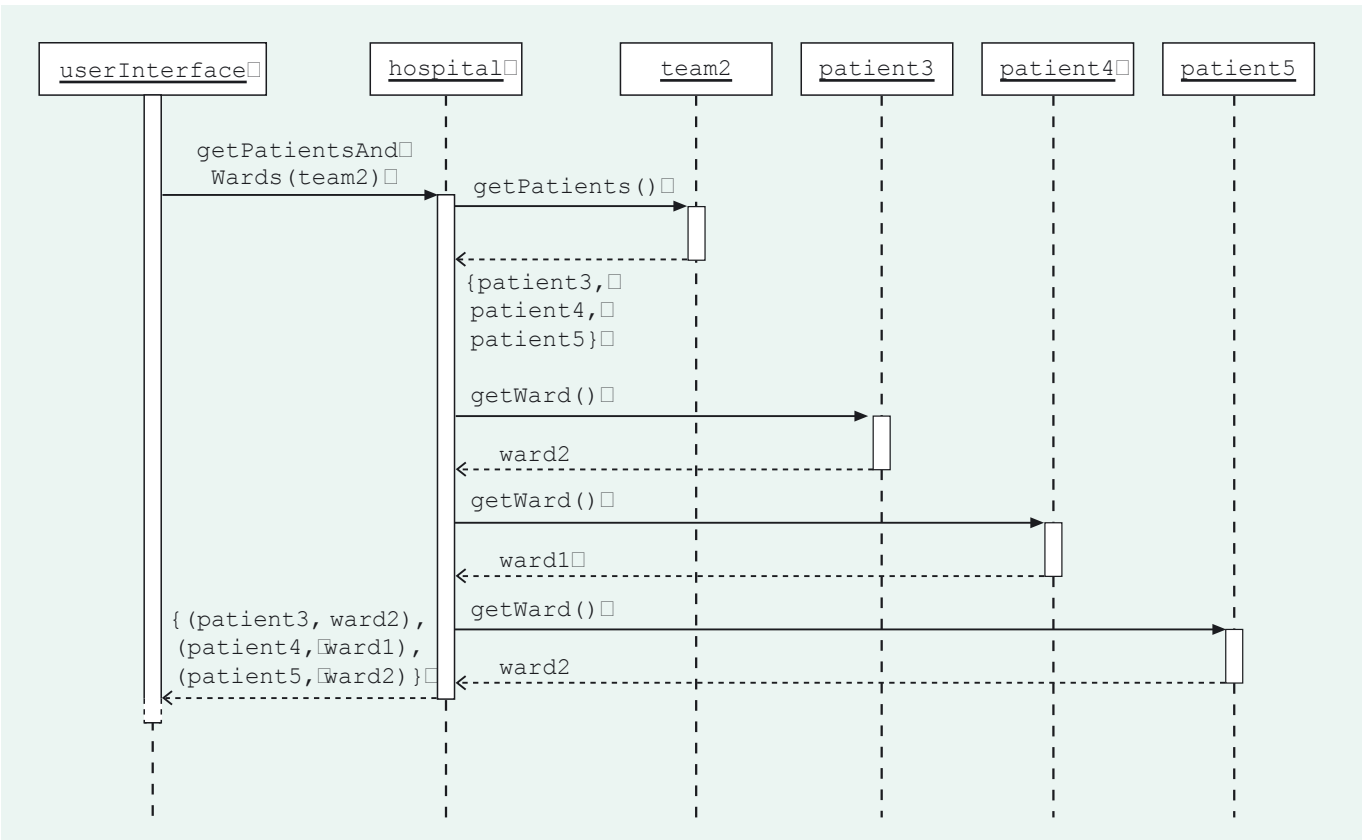

Figure 3 `getPatientsAndWards(aTeam)`: Design 2's sequence diagram

We will shortly compare particular aspects of Designs 1 and 2 in some detail. To prepare for this and help you appreciate the significant differences between the designs, first you will review code that *could* correspond to each of them. Remember that the code that is eventually implemented may be different from the code here, because there are many outstanding design decisions which could influence the details of our implementation.

Code corresponding to Design 1

In *Unit 6*, Subsection 3.2, we presented the following code, which corresponds to Design 1.

```
//Definition of coordinating class
public class HospCoord
{
    //code omitted

    public Map<Patient, Ward> getPatientsAndWards(Team aTeam)
    {
        return aTeam.getPatientsAndWards();
    }

    //code omitted
}

//Definition of Team class
public class Team
{
    //code omitted

    private Collection<Patient> patients; //all the linked Patient objects

    Map<Patient, Ward> getPatientsAndWards()
    {
        Map<Patient, Ward> results = new HashMap<Patient, Ward>();
        for (Patient aPatient : patients)
        {
            results.put(aPatient, aPatient.getWard());
        }
        return results;
    }

    //code omitted
}

//Definition of Patient class
public class Patient
{
    //code omitted

    private Ward ward; //the linked Ward object

    Ward getWard();
    {
        return ward;
    }

    //code omitted
}
```

SAQ 2

The code for Design 1 includes, within the method `getPatientsAndWards()` of the `Team` class, an iteration over a collection of `Patient` objects. For Design 2, in which method would there be a similar iteration?

ANSWER

From the sequence diagram for Design 2 (Figure 3), you can see that it is `hospital` that sends the messages to the `Patient` objects. So the iteration would be in the coordinating method, `getPatientsAndWards(aTeam)`, of `hospital`'s class, `HospCoord`.

Code corresponding to Design 2

Below is code for the classes `HospCoord` and `Team` corresponding to Design 2 (the `Patient` class for Design 1 is the same as for Design 2). Review this code, concentrating on:

- its correspondence with the walk-through and sequence diagram for Design 2;
- the differences between this code and that for Design 1.

```
//Definition of coordinating class
public class HospCoord
{
    //code omitted

    public Map<Patient, Ward> getPatientsAndWards(Team aTeam)
    {
        Map<Patient, Ward> results = new HashMap<Patient, Ward>();
        Collection<Patient> thePatients = aTeam.getPatients();
        for (Patient aPatient : thePatients)
        {
            results.put(aPatient, aPatient.getWard());
        }
        return results;
    }

    //code omitted
}

//Definition of Team class
public class Team
{
    //code omitted

    private Collection<Patient> patients; //all the linked Patient objects

    Collection<Patient> getPatients()
    {
        return patients;
    }

    //code omitted
}
```

Exercise 1

Consider the first step in the walk-through which is:

1 Access the collection of `Patient` objects linked to `team2`, which in this case is {`patient3`, `patient4`, `patient5`}.

For each of Designs 1 and 2:

(a) which object gets access to the collection of `Patient` objects in this step;

(b) what is the code that gives the object in (a) access to the collection of `Patient` objects?

Discussion

(a) In Design 1 it is `team2` that gets access to the collection of `Patient` objects in this step.

In Design 2 it is `hospital` that gets access to the collection of `Patient` objects in this step.

(b) In Design 1 the code is within `Team`'s method `getPatientsAndWards()`. It is the use by the receiver (in this case, `team2`) of its instance variable `patients` within the loop heading:

```
for (Patient aPatient : patients)
```

In Design 2 the code is within `HospCoord`'s method `getPatientsAndWards(aTeam)`. It is the following:

```
Collection<Patient> thePatients = aTeam.getPatients();
```

That is to say, it is the sending of a message to the `Team` object `aTeam` (which in this case is `team2`) to access the value of its instance variable `patients`.

2.2 Design principles

The walk-through steps for Designs 1 and 2 for the *List Team's Patients* use case are the same. The same key things are done within the core system in each case, but the designs differ in which objects do which tasks, and in the message passing between the objects. On what basis might we choose between the designs?

The two designs we are currently considering are actually relatively straightforward. Designs for a large system can involve hundreds of interacting objects. Consequently, comparing the designs, and deciding which will lead to optimum software, can be very challenging indeed. To assist in this, developers use **design principles** – guidelines for which objects should be allocated which kinds of task, and how the objects should communicate. It is important to appreciate that, though design principles can be useful tools, there are no hard and fast rules for design. The relevance of design principles can vary depending on the context; design principles can even conflict in some circumstances, and developers and researchers often disagree about their importance. Learning to apply design principles wisely, knowing which are useful in a certain context, and how to balance them, comes with experience, and you are not expected to become proficient in this from studying M256. Nevertheless, in learning about some basic design principles, you will be gaining a solid foundation from which to begin evaluating key features of designs.

Information experts

In the course so far, we have frequently alluded to the benefits of software objects mirroring real-world entities, whether tangible ones such as hospital patients or intangible ones such as films. This approach guided our early plans for the Hospital System structure: we decided that it would contain, for example, `Patient` objects and `Ward` objects, having attributes and links corresponding to their real-world counterparts. Continuing this approach, by assigning to objects responsibilities that they 'should' have, by analogy with their real-world counterparts, leads to classes and systems that are more easily understood, implemented, maintained and reused.

In Design 1 (Figure 2) the `Team` object is responsible for forming the collection of `Patient` objects and their `Ward` objects; in Design 2 (Figure 3) this is done instead by the coordinating object. It seems intuitively reasonable that a team should be responsible for collecting information about the patients it cares for, so by analogy, the formation of a collection of `Patient` objects and their `Ward` objects is something we might expect a `Team` object to do.

In everyday life, people and other entities undertake tasks related to the information they have. A team provides information about the patients it knows about. So a similar way of thinking about allocating responsibilities is to consider which object most readily has to hand the information needed to carry out the task; that is, which object is the **information expert** for the task. Here we have the task of forming a collection of `Patient` and `Ward` objects. The given `Team` object, `team2`, is linked to the required `Patient` objects, which are linked to the required `Ward` objects. The `Team` object is the information expert for this task, which accords with Design 1, where it is `team2` that forms the collection. In Design 2, the coordinating object, `hospital`, *can* access these objects, but it has to do so via `team2`: `team2` has the information more readily to hand.

Even distribution of responsibilities

Evident from both the sequence diagrams and the code are the facts that in Design 1 most of the work is done by `team2`, whereas in Design 2 most of the work is done by `hospital`, with `team2`'s role being simply to return its `Patient` objects to `hospital`. In fact, it would be possible to design each use case similarly to Design 2, with the coordinating object doing the complex work, merely calling on other objects for simple information. But concentrating the behaviour in one object like this can lead to a large class which is difficult to understand, alter and maintain, and other small classes with minimal functionality, which are poor candidates for reuse. Designs which share responsibilities evenly, with sensible delegation of tasks, are generally preferred.

A common situation involving a choice of how to allocate responsibilities is where an object `p` wants to get an object `r` to do something, but `p` does not have a reference to `r` which would allow `p` to send `r` a message; the object `p` does, however, have a reference to an object `q` which does have a reference to `r`.

This is just the situation we have with `hospital`, `team2` and each of the `Patient` objects. `hospital` is passed a reference to `team2` as an argument to the coordinating message; `team2` has a reference to each of its `Patient` objects; `hospital` wants to get each of the `Patient` objects to reveal its `Ward` object.

The terms **forking** and **cascading** can be useful in comparing the distribution of responsibilities in this kind of design situation.

Forking describes the situation where `p` sends a message to `q` asking it for `r`; `q` returns `r` to `p`, so `p` can then send a message to `r` asking it to perform its task. This is illustrated in the sequence diagram in Figure 4.

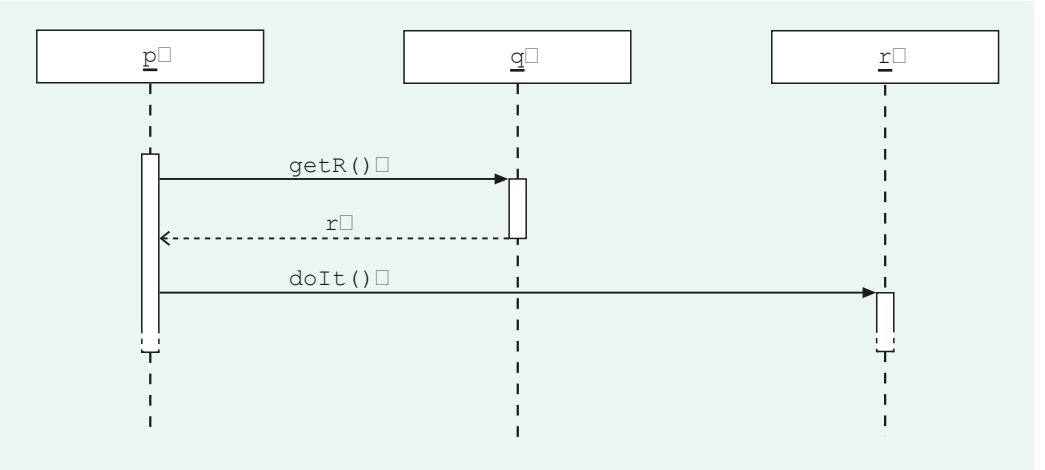

Figure 4 Forking

Designs based on forking tend to have a monolithic, all-powerful object, such as `p` here.

Cascading, on the other hand, involves greater delegation of responsibility. Cascading describes the situation where `p` sends a message to `q`, asking `q` to ask `r` to perform its task; `q` then sends a message to `r` asking it to perform its task. This is illustrated in the sequence diagram in Figure 5.

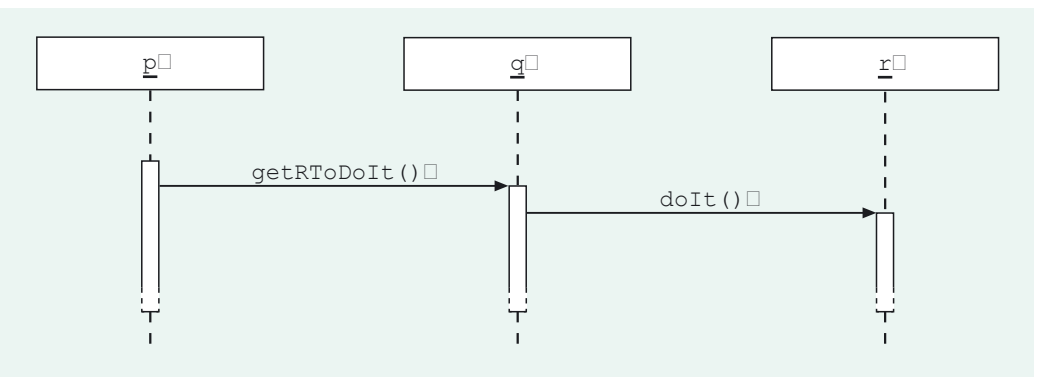

Figure 5 Cascading

A design based on cascading typically divides responsibilities more evenly across classes than one which relies heavily on forking. However, if taken to extremes, cascading can result in unnecessarily lengthy and convoluted code, with objects which are otherwise unconnected with the task at hand acting merely as message passers. Often, designs employing a mixture of forking and cascading are preferred.

SAQ 3

Is Design 2 (Figure 3) based mainly on forking or on cascading?

ANSWER.....

Design 2 is based on forking. In Design 2, `hospital` has no reference to the relevant `Patient` objects. It asks `team2` to return them. It then sends a `getWard()` message directly to each of these `Patient` objects.

Cohesion

Cohesion was introduced in *Unit 5,* Subsection 7.2, as a measure of how strongly related and focused the responsibilities of a component are.

SAQ 4

Which is generally considered preferable: high cohesion or low cohesion?

ANSWER.....

High cohesion is generally preferable.

Generally speaking, the more cohesive a component, the more easily it is reused and maintained. If a component performs unrelated tasks, or does too many tasks, then it is likely to be hard to understand and use, and the chances of its matching the requirements of another software development project are small.

We can consider the cohesion of the individual classes `HospCoord` and `Team`. Collecting together information about patients and their wards 'belongs' with other team-focused behaviour, such as recording which patients are cared for by the team. Allocating this behaviour to `Team` objects, as in Design 1, makes `Team` more cohesive; allocating it to `hospital`, as in Design 2, makes the coordinating class `HospCoord` less cohesive.

Note, though, that a coordinating class is intentionally specific to the particular system for which it is developed, offering controlled access to services performed by the core system. It is therefore not likely to be reused in another context, and its cohesion is not a key concern, though it is of course important for the coordinating class to be maintainable. The coordinating class's lack of reuse potential contrasts with other core system classes, which may conceivably be reused in other contexts; `Patient`, for example, might be used in a variety of health-service systems.

Coupling

In *Unit 5*, Subsection 7.2, you also met the concept of *coupling*. Two classes are coupled when one depends on the other in some way, so requiring the presence of the other. Situations in which there is a dependency between classes `P` and `Q` include the following.

- Objects of class `P` have an instance variable that references an object or objects of class `Q`.
- `P` is a subclass of `Q`.
- An object of class `P` sends a message to an object of class `Q`. That is, inside one of `P`'s methods, a message is sent to an object of class `Q`.
- `P` has a method that receives or returns an object of class `Q`.

Low coupling is usually considered a desirable feature of software. When you are designing a class, the lower its coupling to other classes the more easily reusable and maintainable it is likely to be. Taken to extremes, however, in a system with no coupling, objects would not send messages to each other, contradictory to object-oriented philosophy. Sensible coupling is fundamental to object-oriented software.

There is no absolute measure of coupling, or indeed of cohesion. It is not possible to say, for example, that one design has twice the coupling of another: the concept is not that precise. Instead what is important is to understand where coupling is necessary, where it might be unnecessary, and where it might cause problems.

One issue that occupies developers is striking a balance between coupling and cohesion. At one extreme, low coupling can be achieved by having a few, large, classes, but these classes would tend to have low cohesion. At the other extreme, a system may be fragmented into many small classes, each with high cohesion, but then these classes would be likely to be highly coupled to each other.

SAQ 5

Why might it be more acceptable for the coordinating class to be highly coupled to other classes, than for other core system classes to be highly coupled to one another?

ANSWER.....

It might be more acceptable for the coordinating class to be highly coupled because it is less likely than other core system classes to be a candidate for reuse (as noted above in the discussion of cohesion).

When two classes are associated, it follows that they will be coupled. This is because links of the association between them will be implemented by objects of one, or both, of the classes referencing objects of the other class. When two classes are not associated, they may or may not be coupled. Two non-associated classes may become coupled by virtue of objects of one class sending messages to objects of the other class. Design 2 illustrates this: the classes `HospCoord` and `Patient` are not associated, but in this design `hospital` sends messages to the `Patient` objects.

Later design considerations could lead us to introduce an association between `HospCoord` and `Patient`, in which case we might return to our deliberations here. As we have mentioned, use-case designs are often interdependent – but considering the full complexities of coupling across use cases is beyond the scope of this course

Such a design, in which objects of one class send messages to objects of an non-associated class, tends to increase the coupling in the system. All other things being equal, it can be sensible for objects *only* to send messages to each other when their classes are associated. In Design 1, `team2` sends messages to the `Patient` objects. Since `Team` and `Patient` are associated, in this respect Design 1 is preferable.

Design principles may be related

We have considered some principles that can be helpful in identifying and comparing key features of designs. In fact you have probably noticed that, although these principles provide usefully different perspectives, they are interrelated. The 'information expert' approach, for example, leads us to allocate to `Team` the responsibility for forming the collection of `Patient` and `Ward` objects, in much the same way as considering the cohesion of `Team`.

SAQ 6

Allocation of responsibility *away* from the information expert for a task tends to lead to designs with relatively higher coupling. Why is this?

ANSWER.....

If responsibility is allocated to an object which does not have the required information to hand, then that object has to call on other objects to provide it with the information, thus making it reliant on these other objects, and tending to increase coupling.

For example, if, as in Design 2, `hospital` has the task of getting the `Ward` objects of those `Patient` objects that `team2` (as the information expert) has access to, then the `HospCoord` class is coupled to the `Patient` class as well as to the `Team` class. This is unlike the situation in Design 1, where `HospCoord` is only coupled to `Team`.

Our consideration of design principles so far leads to a preference, at this stage, for Design 1 (our original design) over Design 2. But in reality a software developer would not make decisions in isolation about which design to adopt. Consideration of other use cases' designs might influence the decision, and, similarly, the choice of design for this use case may affect the possibilities for other use cases.

There are other design possibilities for this use case which we will not discuss.

Handling the interrelationships between use cases is challenging and can be complicated, and developing skills in this comes with experience and practice. We have said that we do not expect you to become proficient in this. However, this section has provided you with a basis upon which to differentiate the key features of designs (i.e. consideration of information experts, distribution of responsibilities, forking, cascading, cohesion and coupling), and you will have opportunities to practice comparing designs along these lines later in this unit.

3 Invalid and alternative scenarios

Until now, we have described as 'typical' the scenarios upon which we have based our designs. They embody what you might think of as the usual circumstances in which the user interface will invoke the coordinating method for a particular use care. But a complete design for a coordinating method must describe what should happen in other, less straightforward circumstances. In this section, we consider such circumstances and how a design can be amended or extended to cater for them.

3.1 Valid and invalid scenarios

In *Unit 6*, Subsection 5.1, you saw that a coordinating method may have a pre-condition. Here is the description of the *Treat Patient* use case (labelled B in the requirements document).

> The administrator identifies the patient and the doctor; the doctor must be in the team that cares for the patient. The system records that the doctor has treated the patient.

And here is the coordinating method specification we established for that use case.

```
void recordTreatment(Patient aPatient, Doctor aDoctor)
```

> Pre-condition: `aDoctor` and `aPatient` must be linked to the same `Team` object.
>
> Post-condition: `aDoctor` is linked to `aPatient`.

For this particular use case, the pre-condition also happened to be reinforced by the use case description which states 'the doctor must be in the team that cares for the patient'. In general, pre-conditions on use cases may not be so explicit.

The pre-condition above, restricting the circumstances in which the method should be used, is intended to maintain the following Hospital System invariant (numbered 7 in the Hospital System's initial structural model). It expresses the fact that a doctor who treats a patient must be a member of the team caring for the patient.

> If `aPatient` is any `Patient` object, then any `Doctor` object linked to `aPatient` is also linked to the `Team` object which is linked to `aPatient`.

During the execution of a coordinating method, an invariant may temporarily be broken. Further consideration of this issue is beyond the scope of this unit.

This is something that must always be true of the system's state before and after a coordinating method has completed, which is why it is called an invariant. It is most important that it never becomes false, because if it did it would mean the system's state was invalid. The system would hold information that does not conform to a real-world constraint, and it could no longer be relied upon.

Exercise 2

Which of the object diagrams (a) to (c) of Figure 6 shows the Hospital System in an invalid state with respect to invariant 7 above? Explain your answer.

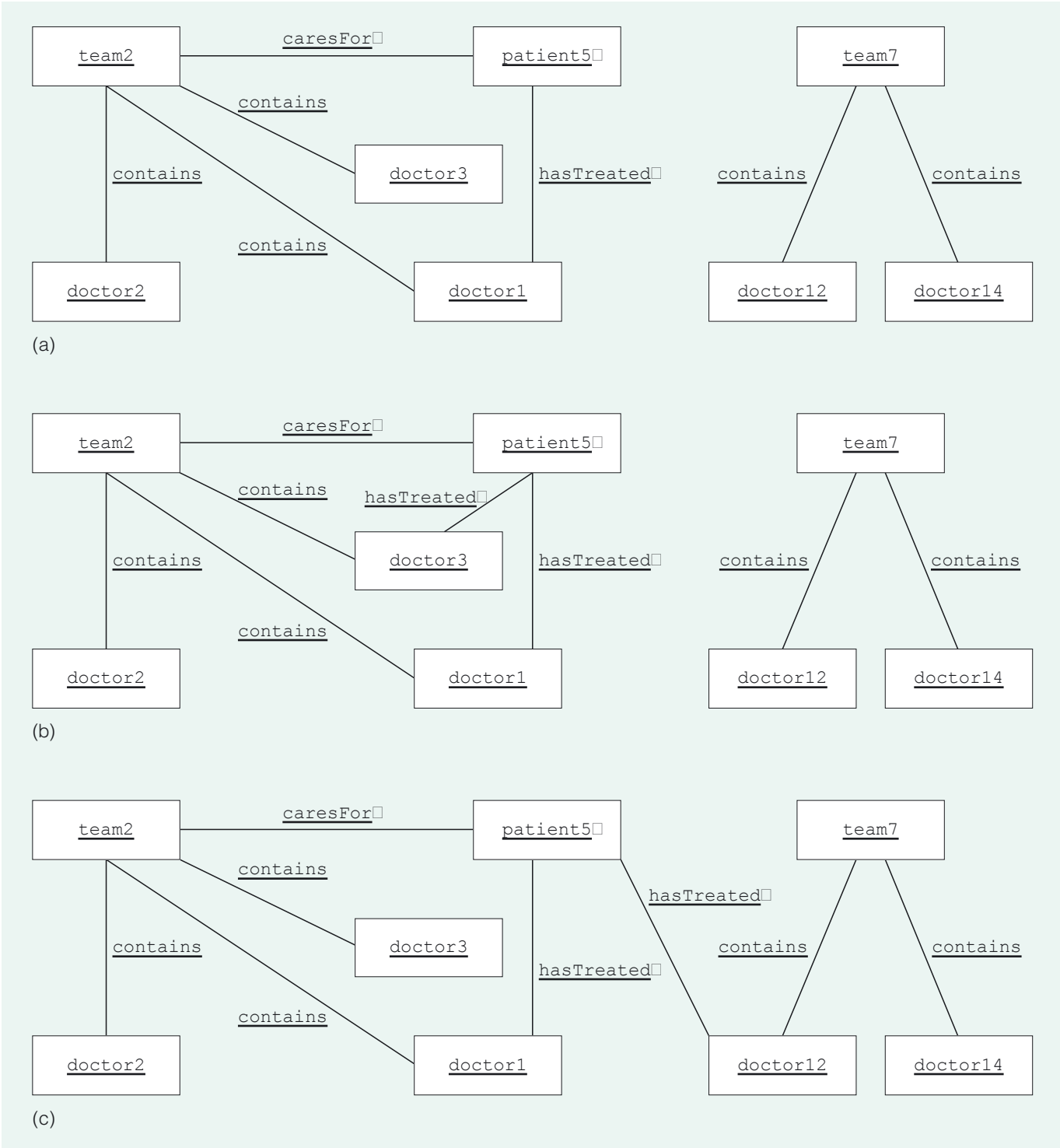

Figure 6 Object diagrams for the Hospital System

Discussion

The object diagrams in (a) and (b) show the system in valid states: invariant 7 is not broken. `patient5` is linked only to `Doctor` objects that are in `patient5`'s team, `team2`.

However, the diagram in (c) shows `doctor12`, who is in `team7`, recorded as having treated `patient5`. The disparity in teams means the system is in an invalid state, because invariant 7 is broken.

Invariant 7, which restricts the `Doctor` objects that can be linked with a given `Patient` object, clearly has implications for the *Treat Patient* use case, which creates a `hasTreated` link between a `Patient` object and a `Doctor` object. This is the reason we gave the *Treat Patient* coordinating method a pre-condition that says the `Doctor` object must be an appropriate one for the `Patient` object.

Unless this pre-condition is met, the user interface should not invoke the coordinating method `recordTreatment(aPatient, aDoctor)`. Later in this unit, you will see how the user interface can be provided with information that it can use to prevent a user requesting the recording of a treatment by a doctor in the wrong team. This is not enough though. Suppose the code in the user interface is faulty, and invokes the `recordTreatment(aPatient, aDoctor)` method with inappropriate `Patient` and `Doctor` objects so that the invariant would be infringed. If the method nevertheless creates the link, the system will be put into an invalid state. Rather than run this risk, we extend the design for the coordinating method so that a check is made *within* the core system that the invariant will not be broken. Only if the check succeeds is the link created.

Valid scenarios

Our design for the coordinating method `recordTreatment(aPatient, aDoctor)` of *Treat Patient* (from *Unit 6*, Subsection 5.1) is, at this stage, based on a scenario involving objects as in the following object diagrams, in Figures 7(a) and (b).

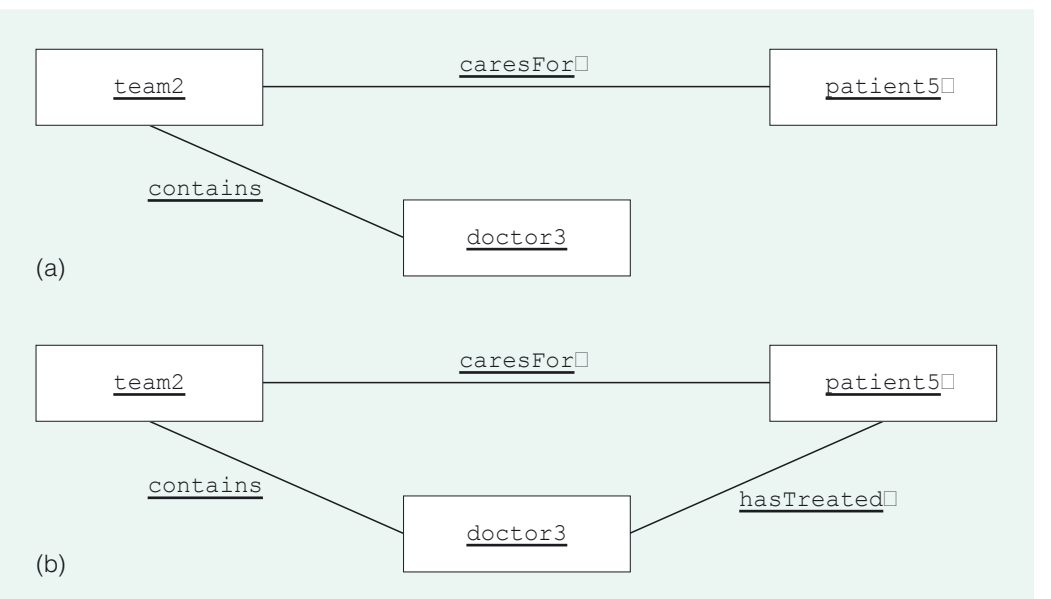

Figure 7 (a) Object diagram for the scenario before *Treat Patient*: `patient5` and `doctor3` are not linked. (b) Object diagram for the scenario after *Treat Patient*: a `hasTreated` link exists between `patient5` and `doctor3`

This is described as a typical scenario, or a **valid scenario**, because the doctor is in the right team: `doctor3` *is* linked to `team2`, the same `Team` object that is linked to `patient5`. Here is the walk-through (it has just one step) and the sequence diagram (Figure 8) previously constructed for *Treat Patient* on the basis of this scenario.

Given: the `Patient` object `patient5`, and the `Doctor` object `doctor3`.

1 `patient5` records a reference to `doctor3`.

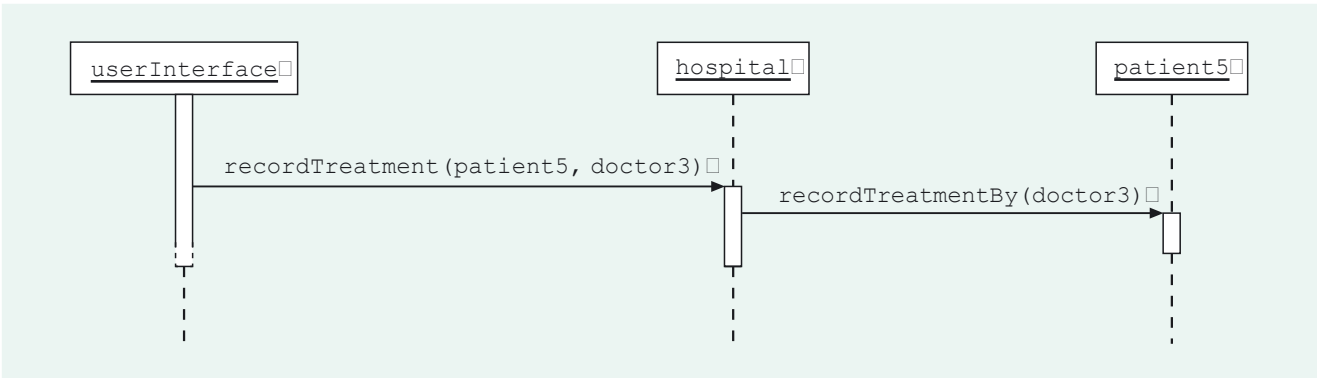

Figure 8 Design 1's sequence diagram for *Treat Patient*

To cater for the possibility that the doctor is not in the right team, a check of this fact must be included, and it should be shown as being carried out successfully in this particular scenario. One way of doing this corresponds to the real-world activity of first finding the patient's team, and then finding the doctors in that team and checking whether the given doctor is one of these doctors. To extend the design to include this check, we will first extend the context of our valid scenario to show other `Doctor` objects linked to `team2`.

There are other ways of making this check which we will not consider here.

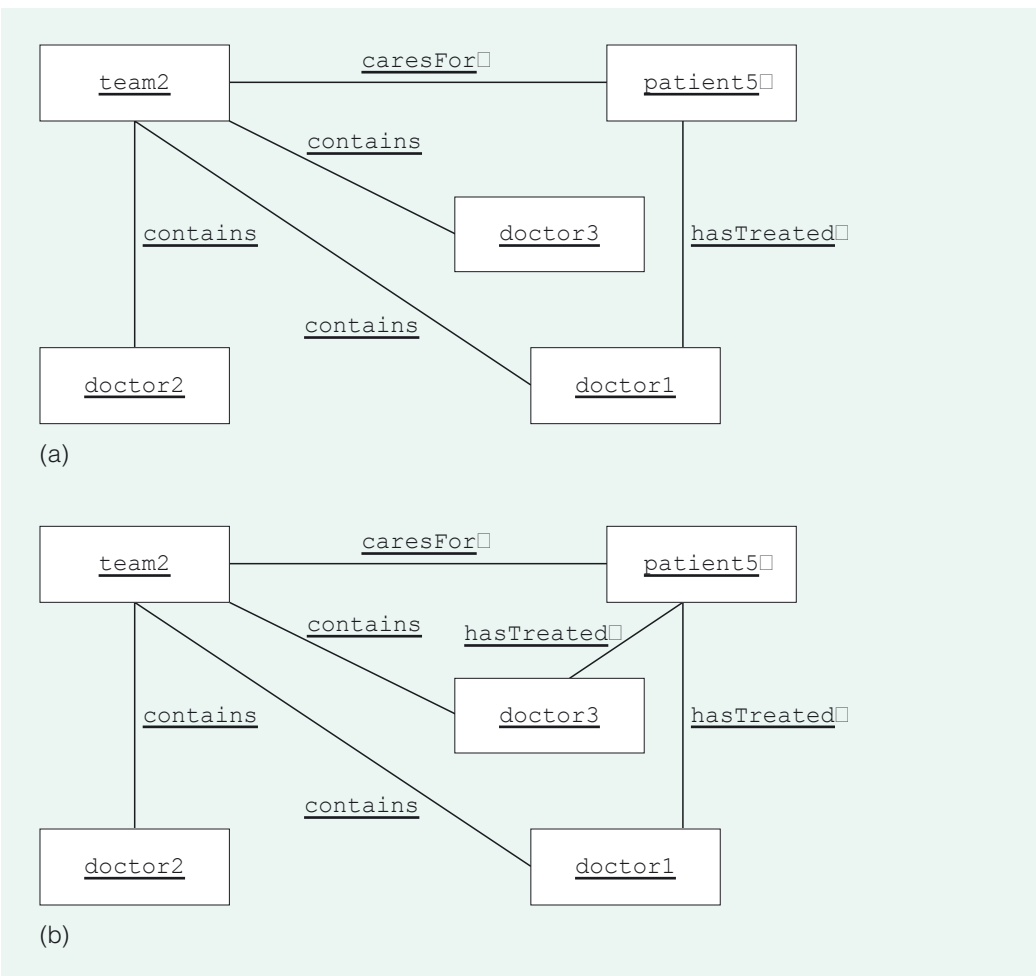

Figure 9 (a) Object diagram for the valid scenario before *Treat Patient*: `patient5` and `doctor3` are not linked. (b) Object diagram for the valid scenario after *Treat Patient*: a `hasTreated` link exists between `patient5` and `doctor3`

Here is a walk-through for the above scenario, based on the idea of checking whether the doctor is in the patient's team.

Given: the `Patient` object `patient5`, and the `Doctor` object `doctor3`.

1 Access the `Team` object linked to `patient5`, which in this case is `team2`.

2 Access the collection of `Doctor` objects linked to `team2`, which in this case is {`doctor1`, `doctor2`, `doctor3`}.

3 Check whether `doctor3` is in {`doctor1`, `doctor2`, `doctor3`}.

As the check succeeds:

4 `patient5` records a reference to `doctor3`.

Because `doctor3` is in the right team, the check at step 3 in the walk-through for our particular scenario is successful, and the next step creates the link between `patient5` and `doctor3`. Figure 10 shows a corresponding sequence diagram – `patient5` sends a message to `team2`, which then checks its collection of `Doctor` objects.

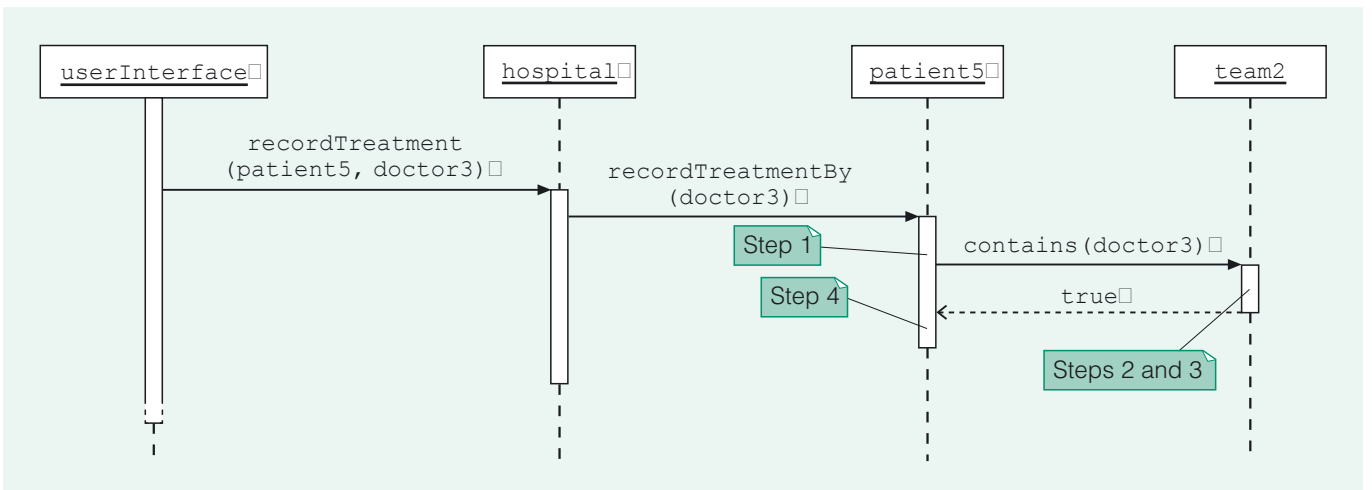

Figure 10 Sequence diagram for *Treat Patient* (valid scenario)

You might have wondered whether the following should be a pre-condition for the coordinating method `recordTreatment(aPatient, aDoctor)`.

`aDoctor` must not already be linked to `aPatient`.

Different treatments of a particular patient by a particular doctor are not distinguished by the system.

In fact there is no need for such a pre-condition, because it is acceptable for the `Doctor` object and the `Patient` object to be linked already. If `patient5` and `doctor3` are already linked, then, since there cannot simultaneously exist two links of an association between two objects, the creation of a link is a redundant activity. That is, if the `Patient` object already has a reference to the `Doctor` object, then step 4, '`patient5` records a reference to `doctor3`', becomes redundant. The coordinating method simply leaves the system state unchanged.

Invalid scenarios

To complete the design, we need to decide what should happen when the doctor is *not* in the right team. To do this requires an **invalid scenario**, which is a scenario in which the pre-condition is not met. We will take as the basis for such a scenario the objects and links shown in the diagram in Figure 11.

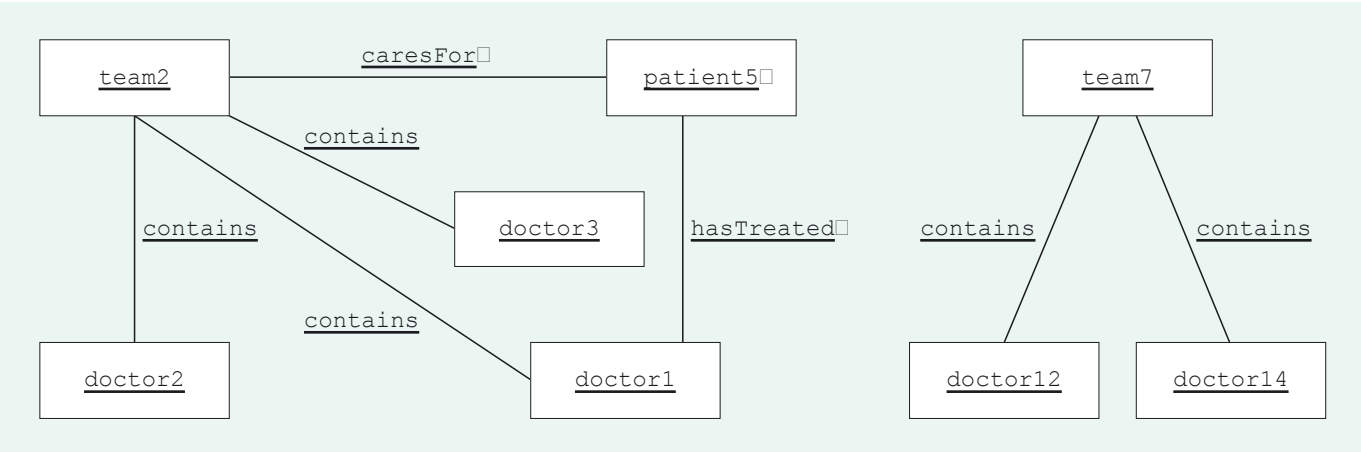

Figure 11 Object diagram for the *Treat Patient* invalid scenario: `patient5` should not be treated by `doctor12`

Because `doctor12` is not in the team that cares for `patient5`, a request to record the treatment of `patient5` by `doctor12` should be denied. That is, invoking `recordTreatment(patient5, doctor12)` should not result in a `hasTreated` link existing between `patient5` and `doctor12`, otherwise the integrity of the core system would be violated. To ensure the link is not created, a simple course of action within the core system is to throw an exception. This will stop processing of the method and allow the user interface to handle the exception, providing the user with some relevant information. We amend the specification of the coordinating method accordingly.

`void recordTreatment(Patient aPatient, Doctor aDoctor)`

Pre-condition: `aDoctor` and `aPatient` must be linked to the same `Team` object; if not, an exception is thrown.

Post-condition: `aDoctor` is linked to `aPatient`.

A walk-through for the invalid scenario should be analogous to that for the valid scenario, except that it should incorporate what happens when the check fails.

Exceptions are Java's particular way of handling various kinds of error situation. In a different language, the programmer might create an `Error` class for this purpose, or the violation might be handled slightly differently, but the overall design would be similar.

Basing the designs for the valid and invalid scenarios on the same general steps, means that, at the detailed design stage, we will be able to cater for different outcomes of the check by using a selection construct.

SAQ 7

Which step(s) in the walk-through for the invalid scenario should differ from those in the valid scenario, and how?

ANSWER.....

For the invalid scenario, it is the final step that should differ, as follows.

As the check fails:

4 throw an exception.

Here is our walk-through for the invalid scenario.

Given: the `Patient` object `patient5`, and the `Doctor` object `doctor12`.

1 Access the `Team` object linked to `patient5`, which in this case is `team2`.

2 Access the collection of `Doctor` objects linked to `team2`, which in this case is {`doctor1`, `doctor2`, `doctor3`}.

3 Check whether `doctor12` is in {`doctor1`, `doctor2`, `doctor3`}.

As the check fails:

4 throw an exception.

The sequence diagram in Figure 12 shows how the walk-through steps can be accomplished in this invalid scenario.

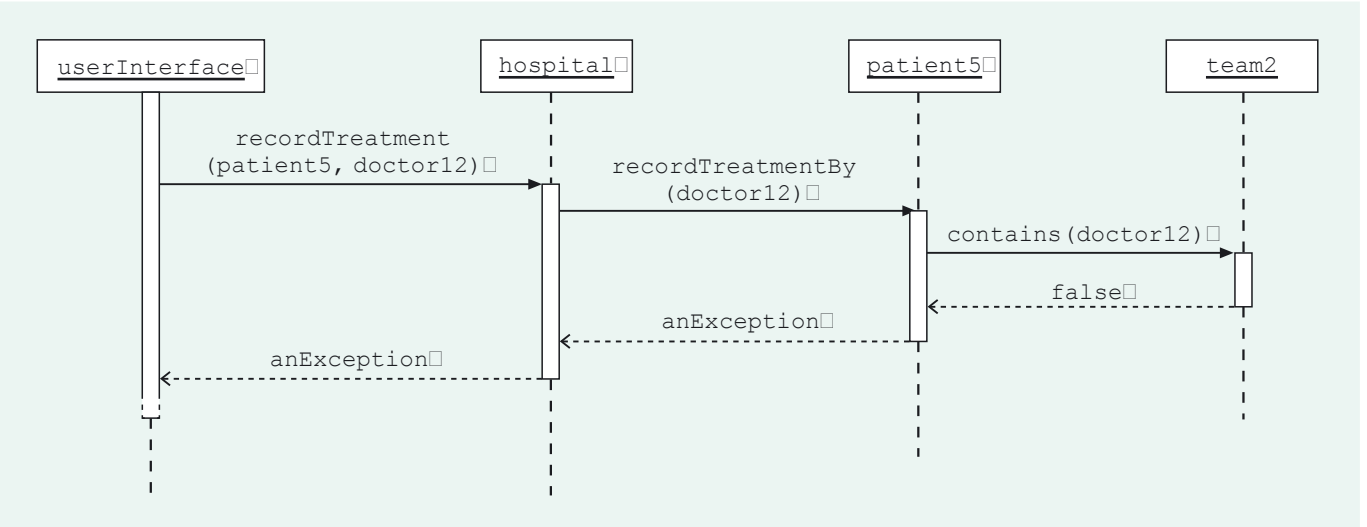

Figure 12 Sequence diagram for *Treat Patient* (invalid scenario)

The sequence diagram shows an exception first being thrown as `patient5`'s response to the `recordTreatmentBy(doctor12)` message, because `patient5` checks whether `doctor12` is in the right team, and the check fails. The exception is propagated back through the system, and an exception is also thrown in response to the coordinating message `recordTreatment(patient5, doctor12)`.

The walk-throughs and sequence diagrams, for both the valid and invalid scenarios now comprise the design for this use case. They are recorded in the developing implementation model, along with consequences for the system structure.

SAQ 8

What method does the design demand of the `Team` class?

ANSWER.....

The design demands a method `contains(aDoctor)`.

In the class `Team` we specify this method as follows.

`boolean contains(Doctor aDoctor)`

Post-condition: returns `true` if `aDoctor` is linked to the receiver, `false` otherwise.

The final consequence of the scenarios we have considered above is to amend the `recordTreatmentBy(aDoctor)` method of `Patient` (as originally specified in Exercise 9 of *Unit 6*), to include a pre-condition which specifies when an exception is thrown.

`void recordTreatmentBy(Doctor aDoctor)`

Pre-condition: `aDoctor` and the receiver must be linked to the same `Team` object; if not, an exception is thrown.

Post-condition: a reference to `aDoctor` is recorded.

SAQ 9

Consider the above walk-throughs for the valid and invalid scenarios. Which association does step 2 (for each walk-through) require to be navigated, and in which direction?

ANSWER.....

The association `contains` must be navigated, in the direction from `Team` to `Doctor`.

Corresponding to the navigation of `contains`, `Team` will have responsibility for holding a link reference.

The next exercise asks you to consider a different way of checking that the doctor is in the right team, thereby giving you practice in comparing designs.

Exercise 3

We will refer to the design so far developed for the *Treat Patient* use case as Design 1. For this exercise you will need to refer to Design 1's walk-through for the valid scenario, and its sequence diagram in Figure 10.

You will compare Design 1 with another design, Design 2, for this same valid scenario. Design 2 has the same walk-through as Design 1, but its sequence diagram is different. It is as follows.

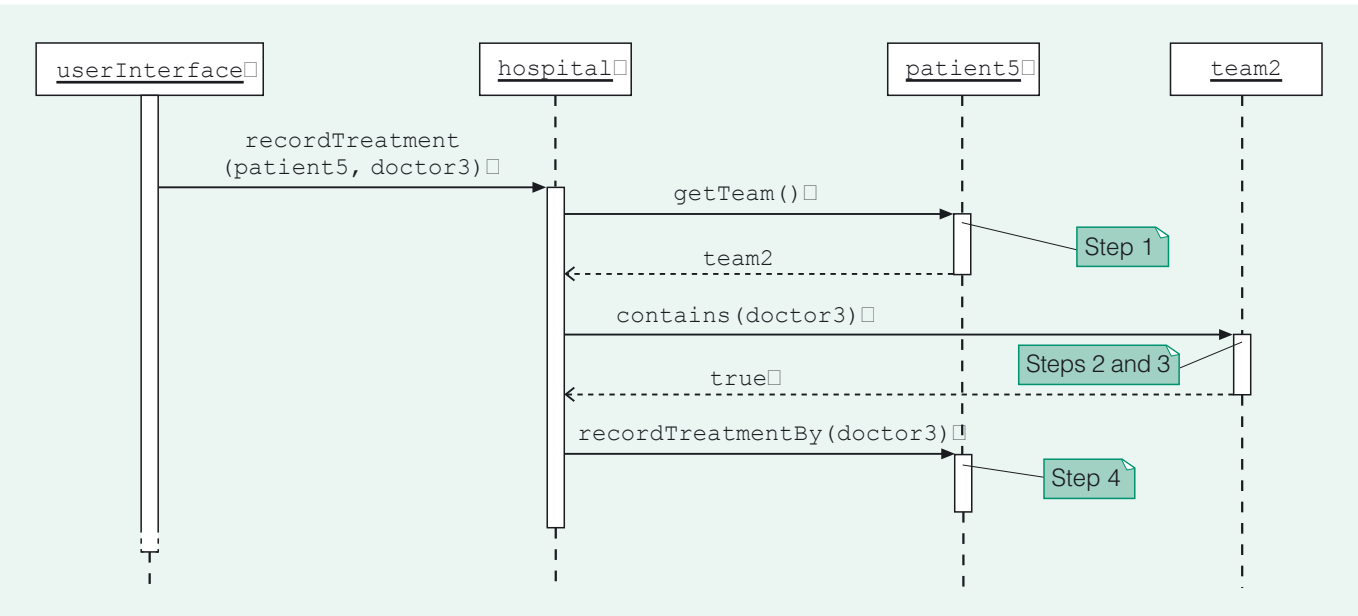

Figure 13 Design 2's sequence diagram for *Treat Patient* (valid scenario)

(a) Briefly describe the interactions between objects of the scenario for each of Designs 1 and 2.

(b) Which design has a more even distribution of responsibilities?

(c) What is the implication of Design 2 on the cohesion and coupling of the class `HospCoord`?

Discussion

(a) In Design 1 the `Patient` object, `patient5`, on receipt of a message from the coordinating object, `hospital`, asks its `Team` object, `team2`, to check whether `doctor3` is one of the `Doctor` objects that `team2` contains. The check succeeds, and `patient5` links itself to `doctor3`.

In Design 2 `hospital` asks `patient5` for its `Team` object; `team2` is returned. `hospital` then asks `team2` to check whether `doctor3` is one of its `Doctor` objects. The check succeeds, and `hospital` tells `patient5` to link itself to `doctor3`.

(b) Design 1 has a more even distribution of responsibilities. In Design 2 `hospital` does most of the work, getting hold of `team2` from `patient5`, then communicating with `team2` to get the check carried out. In contrast, in Design 1, handling the check is delegated to `patient5`.

(c) The cohesion of `HospCoord` is reduced (it is required to carry out more tasks), and its coupling is increased (it is coupled to `Team` as well as `Patient`).

3.2 Alternative scenarios

You have just learnt that a design may be extended to describe how the core system should deal with the possibility of a pre-condition not being met, and an invariant therefore being in jeopardy. A design may also need to be extended to describe an alternative, specified course of action.

To explain, we will consider the *Admit Patient* use case (labelled A in the requirements document). Here is the use case description.

> The administrator provides the system with the patient's name, sex and date of birth and identifies the team that cares for the patient.
>
> The system identifies a ward of the appropriate type with empty beds; if there is a choice of such wards one of those with the largest number of empty beds is chosen.
>
> If there is no appropriate ward with empty beds the administrator is informed of this fact.
>
> If there is an appropriate ward with empty beds the system does the following two things:
>
> 1 It records:
>
> - the patient's name, sex and date of birth;
> - that the patient is under the care of the given team;
> - that the consultant that heads the team is responsible for the patient;
> - that the patient is on the ward;
>
> 2 It informs the administrator of the ward to which the patient has been admitted.

Note that the description specifies both what should happen when there is an appropriate ward with empty beds, and what should happen when there is not such a ward.

Here is the specification from *Unit 6*, Subsection 5.2, for the corresponding coordinating method. In accordance with the use case description, it too describes what should result in both situations.

> `Ward admit(Name aName, Sex aSex, M256Date aDate, Team aTeam)`
>
> Post-condition: if there is no `Ward` object of the appropriate type with at least one free bed then `null` is returned.
>
> Otherwise a new `Patient` object, `aPatient`, is created with the supplied attribute values, and `age` according to `aDate`, and:
>
> (i) `aPatient` is linked to `aWard`, where `aWard` is a `Ward` object of the appropriate type with the greatest number of free beds, and `numberOfFreeBeds` of `aWard` is decremented;
>
> (ii) `aPatient` is linked to `aTeam`;
>
> (iii) `aPatient` is linked to the `ConsultantDoctor` object that is linked to `aTeam`;
>
> (iv) `aWard` is returned.

In our original scenario there *is* a ward of the appropriate type with an empty bed, as illustrated in Figure 14.

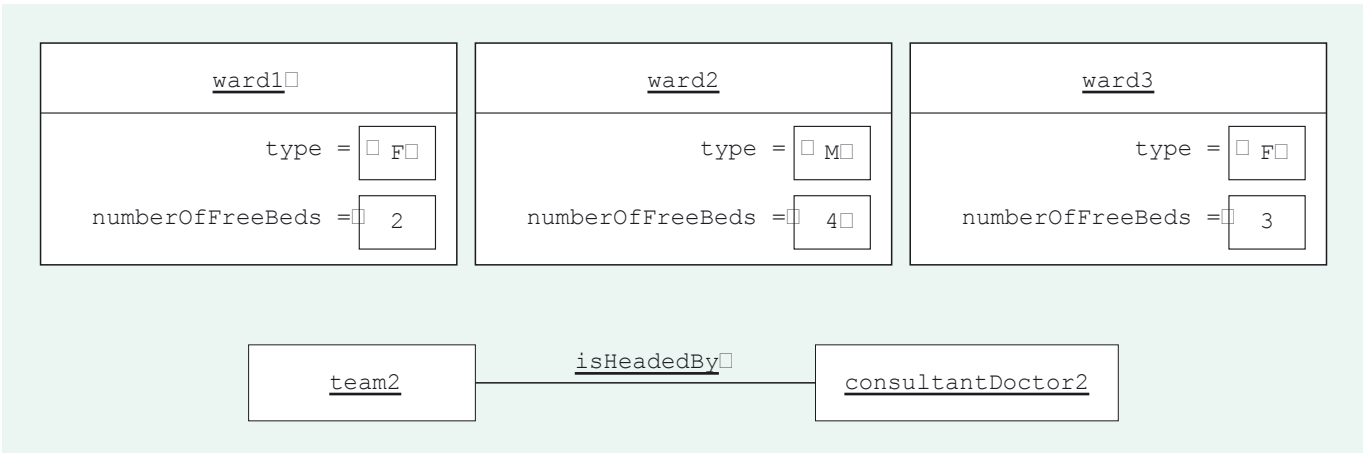

Figure 14 Object diagram for the scenario before *Admit Patient* (a suitable ward exists)

After the admission of the new patient has been recorded, the relevant part of the system is as shown in Figure 15.

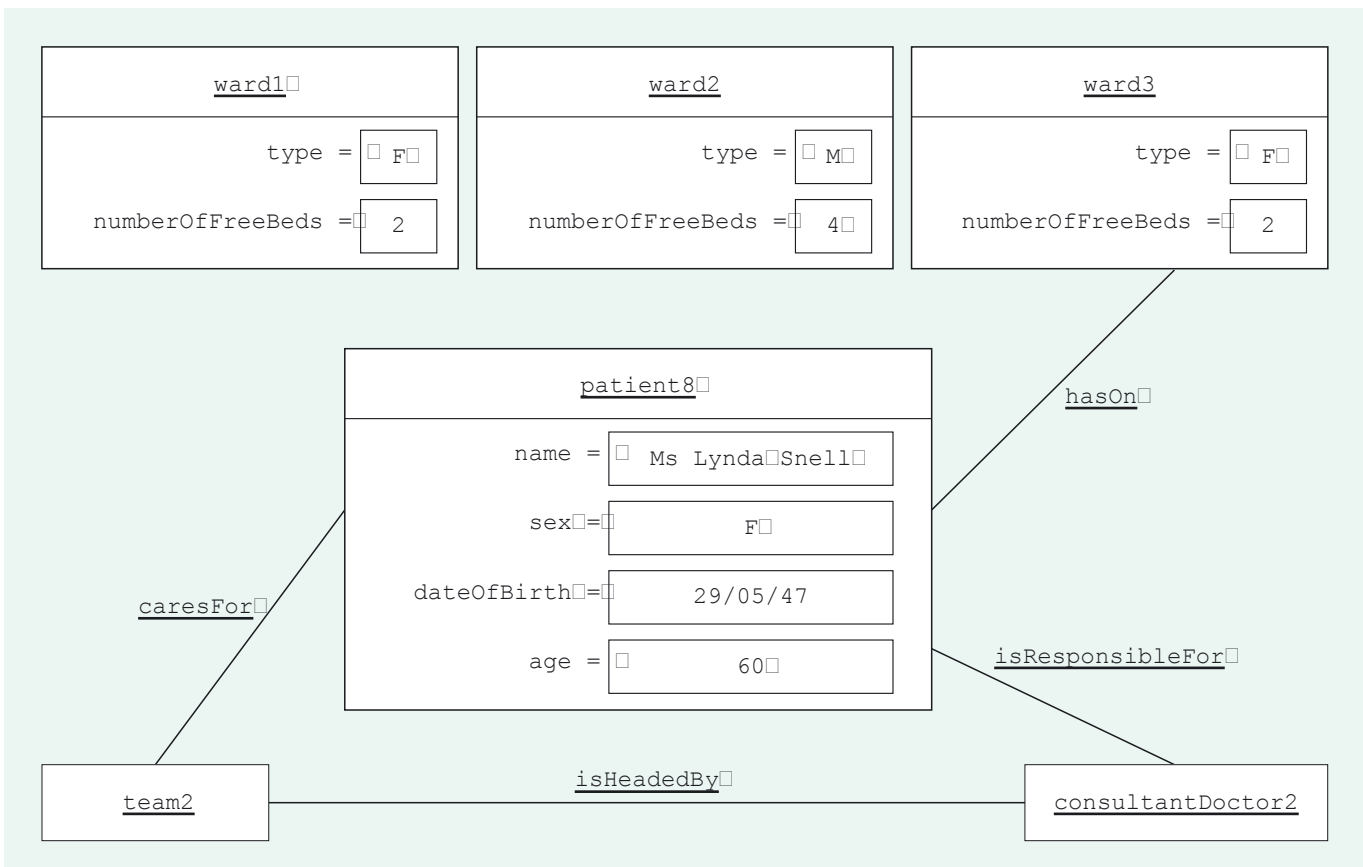

Figure 15 Object diagram for the scenario after *Admit Patient* (a suitable ward exists)

Here is the start of the walk-through we set out for this scenario towards the end of *Unit 6*, Subsection 5.2.

Given: name `Ms Lynda Snell`, sex `F`, date of birth `29/05/47`, and `team2`, a `Team` object.

1a Access the collection of `Ward` objects linked to `hospital`, which in this case is {`ward1`, `ward2`, `ward3`}.

For each of these `Ward` objects:

1b find out its type;

1c if `F`, get its number of free beds.

1d Identify `ward3`, the female `Ward` object with the greatest number of free beds.

2 Create `patient8`, a new `Patient` object with the supplied attribute values and `age` according to the supplied date.

3a `patient8` records a reference to `ward3`.

3b `ward3` records a reference to `patient8`, and reduces `numberOfFreeBeds` by 1.

Steps 1a to 1d of the walk-through are concerned with determining a suitable ward for the patient. If these steps succeed in finding such a ward, as in our scenario, then the `Patient` object and its `hasOn` link with the `Ward` object can be created in the next steps.

However, the use case description makes clear that in general there may be no suitable ward for a patient, in which case the patient should not be admitted. Accordingly, the specification states that in this situation `null` should be returned. Such a situation is a perfectly valid **alternative scenario** for this use case; it should not result in an exception (no pre-condition has been broken) but must be catered for in the design.

Our design needs, therefore, to include a specific alternative scenario in order to illustrate what should happen in this situation. We will suppose that the patient to be admitted has the same details as before, and that there are three wards in the hospital, as illustrated in Figure 16.

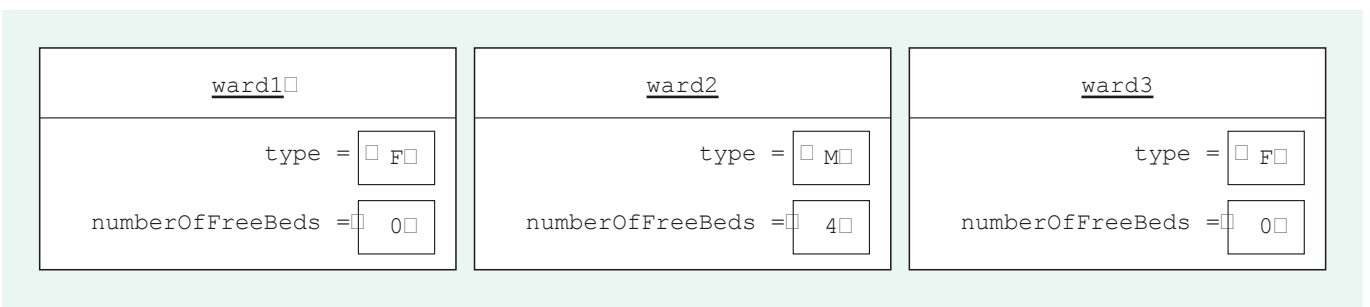

Figure 16 All the female wards are full

As the patient is female, and all the female wards are full, what should happen can be described by the following alternative initial steps.

Given: name `Ms Lynda Snell`, sex `F`, date of birth `29/05/47`, and `team2`, a `Team` object.

1a Access the collection of `Ward` objects linked to `hospital`, which in this case is {`ward1`, `ward2`, `ward3`}.

For each of these `Ward` objects:

1b find out its type;

1c if `F`, get its number of free beds.

As there is no `Ward` object with a free female bed:

1d return `null`.

Exercise 4

Create a sequence diagram corresponding to the above walk-through.

Discussion

Our sequence diagram is shown in Figure 17.

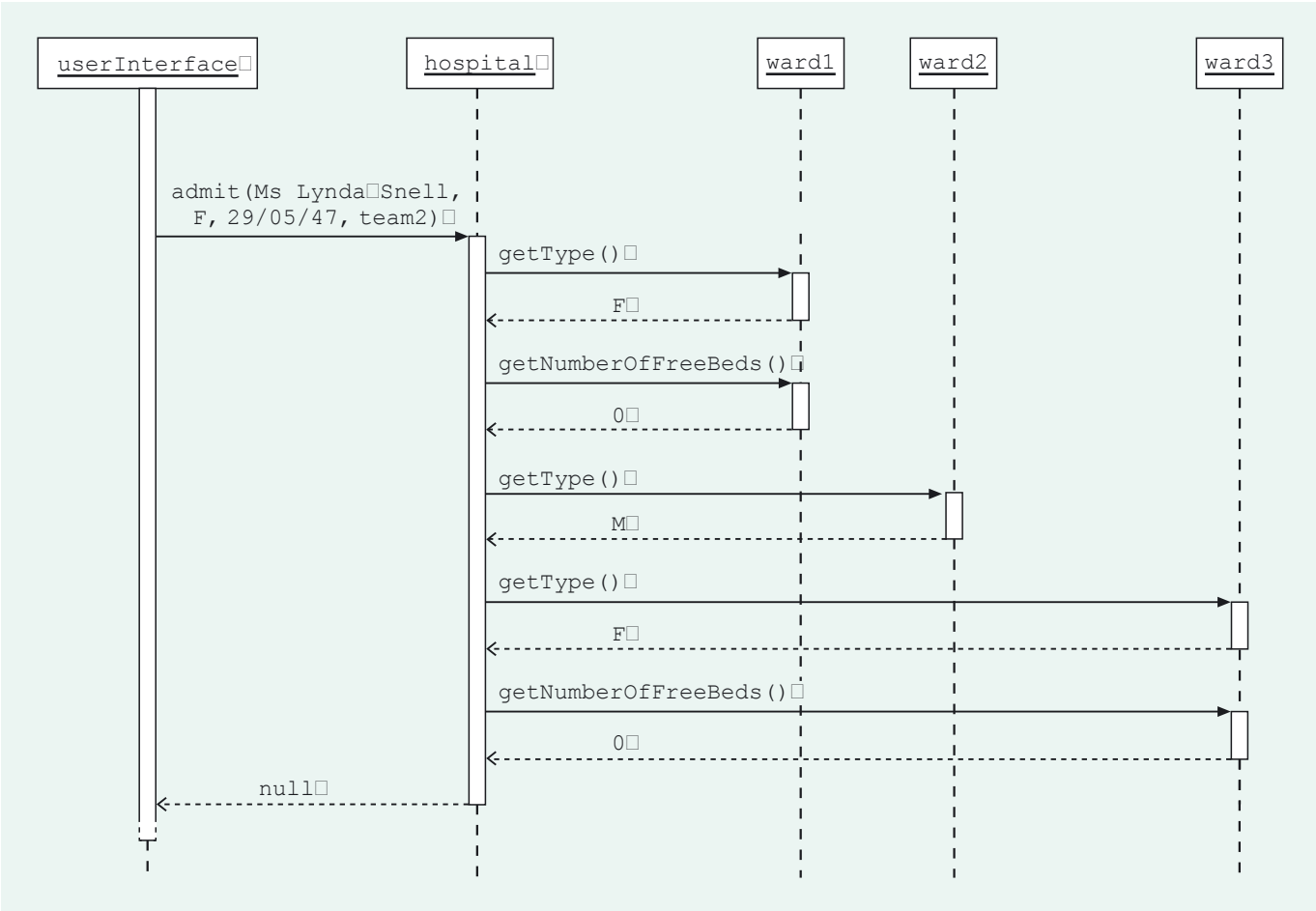

Figure 17 Sequence diagram for *Admit Patient* (alternative scenario)

By including this walk-through and sequence diagram for the alternative scenario, the design for *Admit Patient* is extended to describe what happens within the core system in both situations covered by the use case; that is, both when there is a suitable ward for the patient and when there is not. Note that the design for this alternative scenario does not require any additions to the structural model: there are no additional methods or navigations.

SAQ 10

Explain the difference between an alternative scenario and an invalid scenario. How should each be dealt with in a design?

ANSWER.....

An alternative scenario is one that is covered in the description of a use case, where it is stated what should happen in two (or more) distinct circumstances. This can be dealt with by checking at an appropriate point in the walk-through so that the relevant steps are taken to deal with the alternative scenario(s).

An invalid scenario is one for which a pre-condition is false. The situation is not covered by the description of the use case, and there is no guarantee of what will happen. It is best to deal with such situations by doing nothing that will compromise the integrity of the core system. Instead an exception should be thrown.

In this section, you have seen how a design can be extended to cover both an alternative scenario and an invalid scenario. Next we will consider whether derived attributes and associations should be implemented, and how such decisions affect the designs.

4 Derived attributes and associations

Derived attributes and associations were first introduced in *Unit 3*, Section 7.

The structural model for the Hospital System denotes some elements as *derived*, using a slash, /. Such elements are attributes and associations which represent information that can be deduced from other aspects of the model.

The real-world concepts corresponding to derived elements are significant; that is why these elements are included in the structural model. However, if the system has an alternative means of obtaining the information represented by a derived element, the element may not actually need to be implemented in the system. The designs so far have assumed that all elements *will* be implemented; now it is time to decide whether or not this will be the case and, if a particular attribute or association is not implemented, to determine the consequences on the designs.

4.1 Derived attributes

In general, the attributes that are assigned to classes during development will be implemented in the code as instance variables. However, consider `numberOfFreeBeds`, an attribute of a `Ward` object. This is marked as a derived attribute in the structural model, by being prefaced by a slash (see Section 4 of the *Handbook*). The number of free beds on a ward can be calculated by subtracting the number of patients on the ward from the capacity of the ward. The relationship between number of patients, number of free beds and capacity, which is the basis of this calculation, is expressed by the following invariant (number 3) on the structural model.

> For each `Ward` object, the value of its `numberOfFreeBeds` attribute is equal to the value of its `capacity` attribute minus the number of `Patient` objects to which it is linked.

It follows that the attribute `numberOfFreeBeds` may not need to be implemented as an instance variable of `Ward`, since it represents information that might be found in another way. To see how the system might do this, suppose we decide to implement `Ward` without an instance variable `numberOfFreeBeds`, and that `ward9` is a `Ward` object as illustrated below.

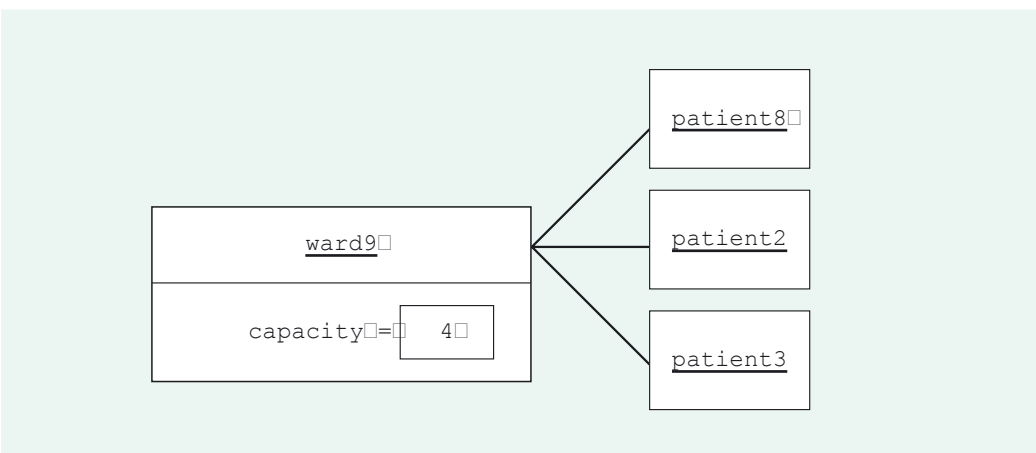

Figure 18 `ward9` and its `Patient` objects

To find the number of free beds on `ward9`, the system must be able to find out the number of `Patient` objects linked to `ward9`, and then subtract this number from `ward9`'s capacity.

SAQ 11

What is the value of `ward9`'s `numberOfFreeBeds` attribute when the system is in the state represented by Figure 18?

ANSWER.....

The value of the `numberOfFreeBeds` attribute is 1 (i.e. a `capacity` of 4 minus the 3 links to `Patient` objects).

To perform this calculation the system needs some way to determine the number of `Patient` objects linked to `ward9`. Our designs already enable this to be done because we previously specified the following instance variable of `Ward` (as part of the design for the *List Ward's Patients* use case in *Unit 6*, Subsection 3.1).

`Collection<Patient> patients`

References a collection of all the linked `Patient` objects.

`ward9` has an instance variable, `patients`, which references a collection of its linked `Patient` objects. The `Ward` object can access this collection and thereby find its size, that is, the number of linked `Patient` objects.

Hence, whenever the system needs to find the number of free beds on a ward, instead of retrieving this number as the value of an instance variable of the `Ward` object, it can perform a simple calculation. This means that `numberOfFreeBeds` need not be implemented as an instance variable. Note that `numberOfFreeBeds` would nevertheless remain, from the perspective of a client of the core system, an attribute of a `Ward` object. Whether or not this attribute is implemented as an instance variable is of no concern outside the core system.

But what would be the benefit of not implementing `numberOfFreeBeds` as an instance variable? Why go to the trouble of performing a calculation? The answer lies in the consistency of the system. If `numberOfFreeBeds` is implemented as an instance variable, then any invariant involving this instance variable must always be true. Whenever a patient is added to or removed from a ward, not only does the system have to create or destroy a `Patient` object's links with a `Ward` object, but it also has to update that `Ward` object's `numberOfFreeBeds` instance variable in order to maintain invariant 7, upon which the derivation is based.

This may not be a significant extra load for our relatively simple Hospital System, but in a larger, more complex system the overheads involved in maintaining invariants can be extensive. Often, derived elements are deemed too costly to maintain, and are not implemented.

In a software development project, considerable time may be spent comparing the work involved in maintaining a derived attribute against the work involved in performing a calculation whenever the attribute value is required. We do not expect you to decide whether or not to implement derived attributes, just that you should be able to identify the effect on a particular design of not implementing a derived attribute. We will illustrate this by considering what happens to the design for *Admit Patient* if we choose *not* to implement `numberOfFreeBeds` as an instance variable.

Here is a walk-through we originally created (in *Unit 6*, Subsection 5.2) for *Admit Patient*, for the scenario illustrated in Figures 14 and 15.

Given: name `Ms Lynda Snell`, sex `F`, date of birth `29/05/47`, and `team2`, a `Team` object.

1a Access the collection of `Ward` objects linked to `hospital`, which in this case is {`ward1`, `ward2`, `ward3`}.

For each of these `Ward` objects:

1b find out its type;

1c if `F`, get its number of free beds.

1d Identify `ward3`, the female `Ward` object with the greatest number of free beds.

2 Create `patient8`, a new `Patient` object with the supplied attribute values and `age` according to the supplied date.

3a `patient8` records a reference to `ward3`.

3b `ward3` records a reference to `patient8`, and reduces `numberOfFreeBeds` by 1.

4a `patient8` records a reference to `team2`.

4b `team2` records a reference to `patient8`.

5 Access the `ConsultantDoctor` object linked to `team2`, that is, `consultantDoctor2`.

6 `patient8` records a reference to `consultantDoctor2`.

7 Return `ward3`.

The sequence diagram in Figure 19 accompanied this walk-through.

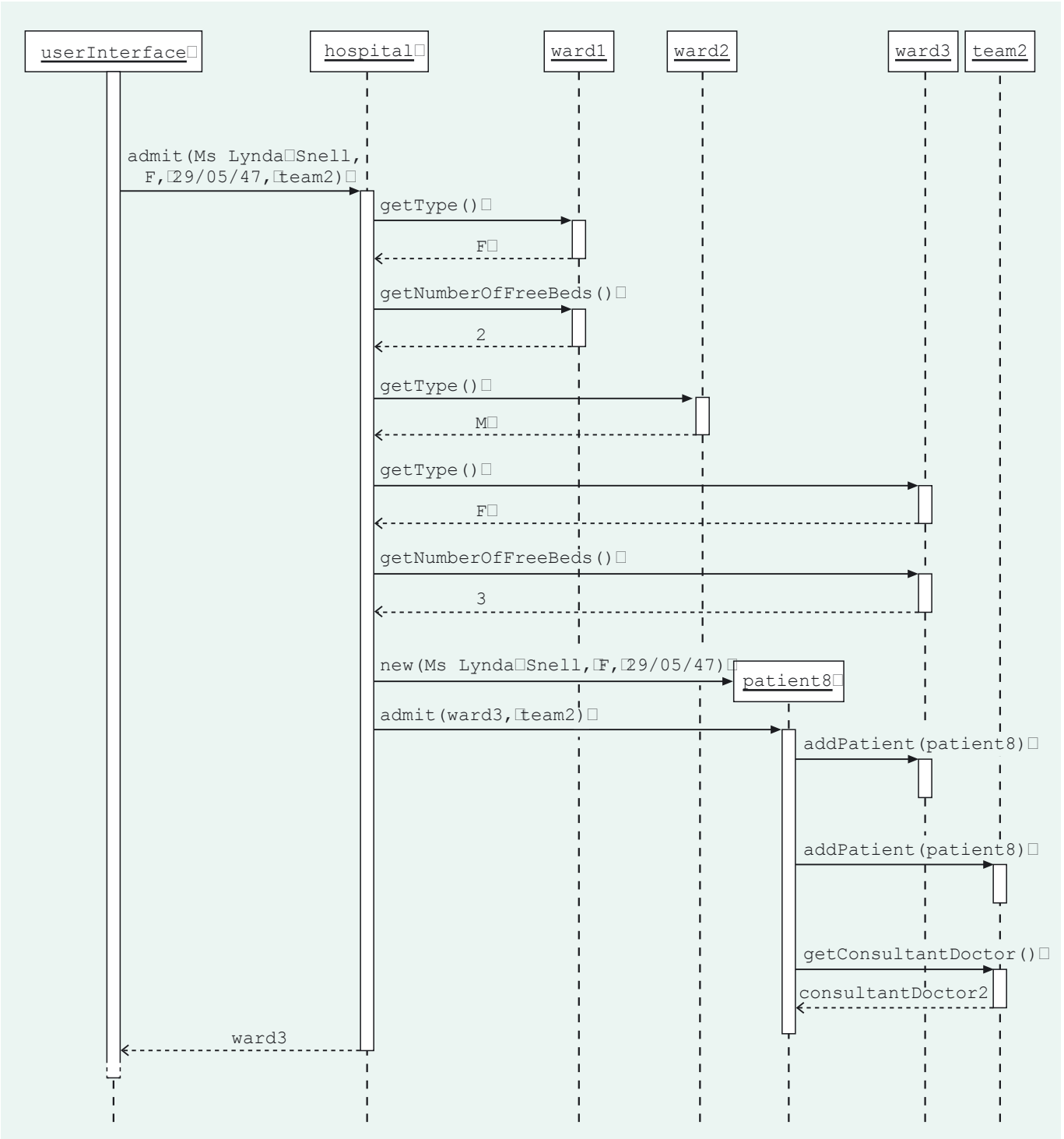

Figure 19 Sequence diagram for *Admit Patient* (valid scenario)

In accordance with this design, the following method for `Ward` was specified at the end of *Unit 6*, Subsection 5.2.

`int getNumberOfFreeBeds()`

Post-condition: returns `numberOfFreeBeds`.

If there is no `numberOfFreeBeds` instance variable, then there is no need to change this specification. To a client sending a message `getNumberOfFreeBeds()`, the *effect* is still to return the value of the attribute `numberOfFreeBeds`, just as specified. What changes is *how* this method works. Instead of simply returning the value of a `numberOfFreeBeds` instance variable, the receiver `Ward` object will access the value of its `capacity`

instance variable, and subtract from this value the size of its `patients` collection, as per invariant 7.

Clearly then, if the attribute `numberOfFreeBeds` is not implemented as an instance variable, then what is required for the method `getNumberOfFreeBeds()` becomes more complex. On the other hand, linking a new `Patient` object to a `Ward` object is simpler if there is no `numberOfFreeBeds` instance variable to update. Having linked `patient8` to `ward3`, there is no need to decrement `numberOfFreeBeds`, so step 3b becomes simply:

3b `ward3` records a reference to `patient8`.

Before deciding whether to implement `numberOfFreeBeds` as an instance variable, the effects on the designs for all use cases would, in reality, be evaluated and compared. We will not do this here, but simply state that we will choose *not* to implement `numberOfFreeBeds` as an instance variable. To record this decision we enclose `numberOfFreeBeds` in brackets in the implementation model's description for `Ward`, as follows.

See Section 5 of the *Handbook*.

Class `Ward`	A hospital ward
Attributes	
`String name`	The unique name of the ward
`Sex type`	Whether the ward is for male or female (M or F) patients
`int capacity`	The maximum number of patients that can be on the ward at any one time
[`int /numberOfFreeBeds`	The number of free beds on the ward]

SAQ 12

Look at the initial structural model for the Hospital System in Section 4 of the *Handbook*. Which other attributes are derived?

ANSWER.....

The attributes `sex` and `age` of `Patient` are derived.

We will choose to implement `sex` as an instance variable, but not `age`. So the description of the `Patient` class in the implementation model is amended as follows.

Implementing `sex` as an instance variable simplifies, for example, the *Transfer Patient* use case. We will not justify these decisions further.

Class `Patient`	A patient in the hospital
Attributes	
`Name name`	The name of the patient
`Sex /sex`	The sex (M or F) of the patient
`M256Date dateOfBirth`	The date of birth of the patient
[`int /age`	The age of the patient in years]

Note that because we have decided to implement `sex` as an instance variable, it appears unchanged in the `Patient` class description.

SAQ 13

Here is the specification for a constructor of the class `Patient`, arising from our original design for *Admit Patient*.

`Patient(Name aName, Sex aSex, M256Date aDate)`

Post-condition: initialises a new `Patient` object with the given attribute values and `age` according to `aDate`.

Does this specification need to change now that we have decided not to implement `age` as an instance variable? Explain your answer.

ANSWER.....

The specification does not need to change. To a client, a `Patient` object still has an `age` attribute, regardless of how it is implemented. Invoking the constructor should appear to set this attribute.

4.2 Derived associations

For our systems, we plan that associations will generally be implemented by objects of one or both classes at the ends of the association holding references to objects of the class at the other end. However, just as with derived attributes, a derived association represents information that the system could determine in an alternative way. The developer needs to compare the relative merits of implementing and not implementing a derived association. In this subsection, you will learn to identify the effect on a particular design of not implementing an association, but again we do not expect you to perform a full evaluation of the consequences.

Exercise 5

Look again at the initial structural model for the Hospital System in the *Handbook* (Section 4).

(a) Which association is marked as derived?

(b) Consider the walk-through for *Admit Patient*, discussed in Subsection 4.1 above. How would this walk-through be affected if the association in part (a) were not implemented?

Discussion.....

(a) The association `isResponsibleFor`, between `Patient` and `ConsultantDoctor`, is marked as derived.

(b) If `isResponsibleFor` were not implemented, then there would be no need to implement a link from a `Patient` object to a `ConsultantDoctor` object. The following steps, which are concerned with creating this link, would simply be removed from the walk-through.

5 Access the `ConsultantDoctor` object linked to `team2`, that is, `consultantDoctor2`.

6 `patient8` records a reference to `consultantDoctor2`.

It is clear from Exercise 5 that *not* implementing `isResponsibleFor` makes the design for *Admit Patient* simpler. But let us now consider the *List Patient's Treatment* use case (labelled G in the requirements document). Here is the specification for one of the three coordinating methods for this use case, with a walk-through and a sequence diagram for a particular scenario. We will consider the effect on this design of not implementing the `isResponsibleFor` association.

Recall that in *Unit 6*, Subsection 3.3, we partitioned what was required for *List Patient's Treatment* into three coordinating methods.

Coordinating method:

`ConsultantDoctor getConsultantDoctor(Patient aPatient)`

Post-condition: returns the `ConsultantDoctor` object linked to `aPatient`.

Figure 20 shows an object diagram illustrating the scenario.

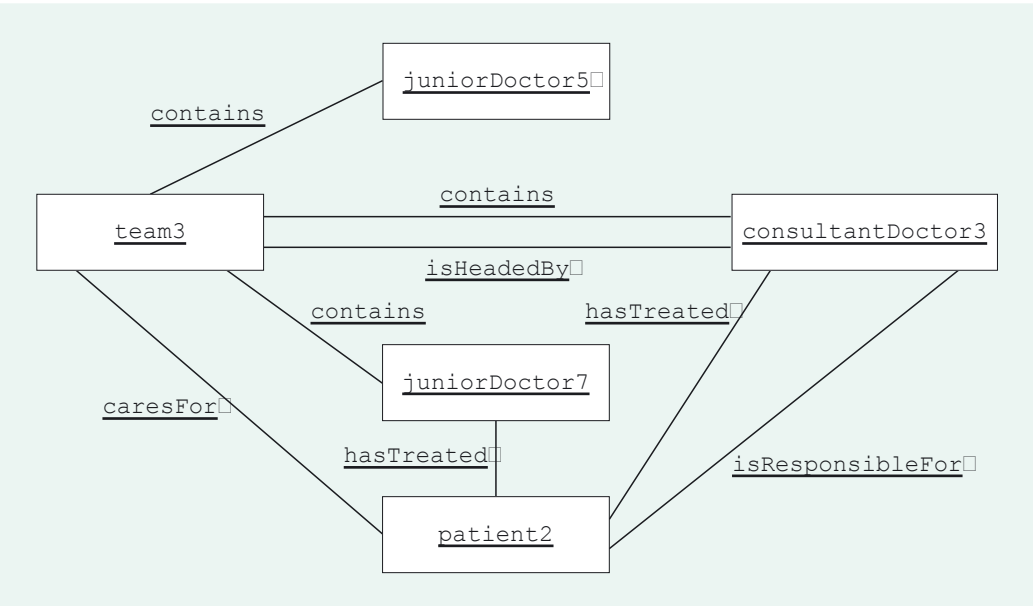

Figure 20 Object diagram for *List Patient's Treatment*

A walk-through for this scenario is as follows:

Given: the `Patient` object `patient2`.

1 Access the `ConsultantDoctor` object linked to `patient2`, which in this case is `consultantDoctor3`.

2 Return `consultantDoctor3`.

Figure 21 shows a sequence diagram for this walk-through.

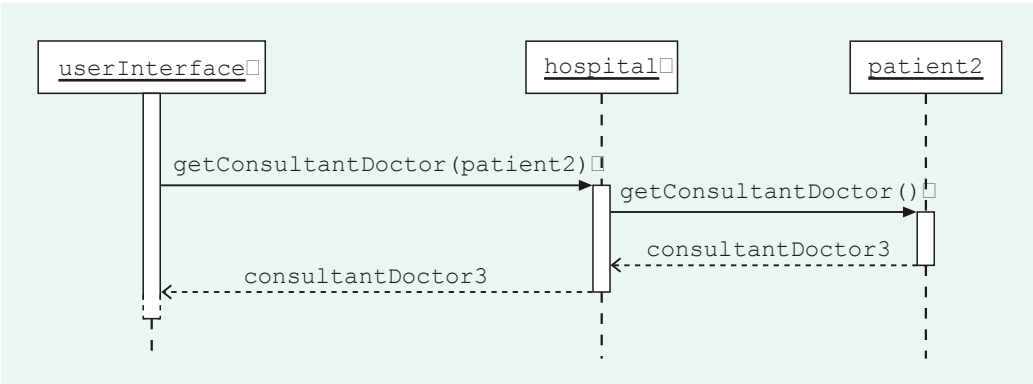

Figure 21 Sequence diagram for `getConsultantDoctor(aPatient)`

This design requires isResponsibleFor to be navigated from Patient to ConsultantDoctor: the ConsultantDoctor object linked to the Patient object must be found. How can this be done if such links are not implemented? The following invariant (number 6), upon which the derivation of isResponsibleFor is based, provides the answer.

> The ConsultantDoctor object linked to any Patient object via isResponsibleFor is linked via isHeadedBy to the Team object to which that Patient object is linked.

Exercise 6

Suppose it is decided *not* to implement the association isResponsibleFor. Using Figure 20, explain how the system could locate the ConsultantDoctor object responsible for patient2.

Discussion

The ConsultantDoctor object responsible for patient2 could be located in two steps.

1 Use the caresFor association to find the Team object, team3, corresponding to the team that cares for the patient.

2 Use the isHeadedBy association to find the ConsultantDoctor object, consultantDoctor3, corresponding to the head of that team.

The invariant guarantees that the ConsultantDoctor located in this way is the one responsible for patient2.

If isResponsibleFor is not implemented, then the accessing of the ConsultantDoctor object needs to take the two-step route described in Exercise 6. The design must be amended as follows.

Step 1 requires navigation of caresFor from Patient to Team.

Step 2 requires navigation of isHeadedBy from Team to ConsultantDoctor.

Neither of these navigations is new: each is already required by other use cases.

> Given: the Patient object patient2.
>
> 1 Access the Team object linked to patient2, which in this case is team3.
>
> 2 Access the ConsultantDoctor object linked to team3, which in this case is consultantDoctor3.
>
> 3 Return consultantDoctor3.

Figure 22 shows a corresponding sequence diagram.

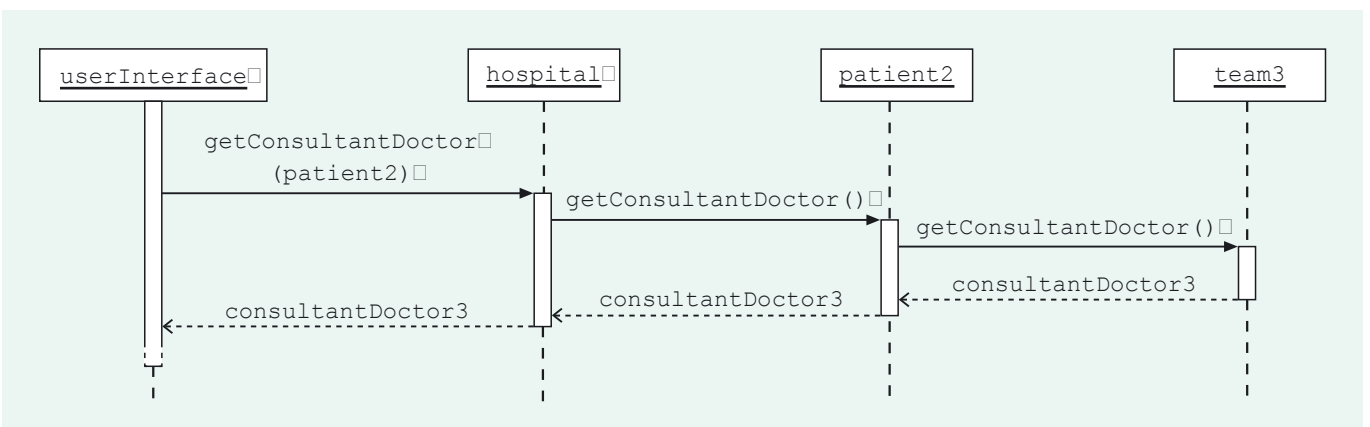

Figure 22 A new sequence diagram for getConsultantDoctor(aPatient)

We will recap and extract some general points from this example. Not implementing a derived association (we looked at `isResponsibleFor`) simplifies designs which involve creating or removing links of the association (we looked at *Admit Patient*). On the other hand, not implementing the association requires extra work whenever that association is navigated (we considered the navigation of `isResponsibleFor` in *List Patient's Treatment*). By weighing up such facts, we could decide whether or not to implement the association.

However, in this case there is another factor to consider. Not only can `isResponsibleFor` be derived from `isHeadedBy` and `caresFor`, but, as discussed in *Unit 3*, `caresFor` can be derived from `isHeadedBy` and `isResponsibleFor`. The decision to denote `isResponsibleFor` as derived in the initial structural model was rather arbitrary: we could equally well have denoted `caresFor` as derived.

Consider the scenario whose objects are illustrated in Figure 20 above. Suppose we decide to implement `isResponsibleFor` but not `caresFor`. How else, other than by using the `caresFor` association, could the `Team` object linked to `patient2` be determined? Well, first `isResponsibleFor` could be navigated to find the `ConsultantDoctor` object linked to `patient2`, i.e. `consultantDoctor3`, and then `isHeadedBy` could be navigated to find the `Team` object linked to this `ConsultantDoctor` object, i.e. `team3`.

Clearly one of `caresFor` and `isResponsibleFor` must be implemented, since the derivation of each requires the other. But which one should be implemented?

In the following exercises you will investigate the effect of *not* implementing the `caresFor` association.

Exercise 7

Consider the walk-through for *Admit Patient* given in Subsection 4.1 above.

(a) Which steps would be simplified if `caresFor` were not implemented?

(b) Which steps would be made more complicated if `caresFor` were not implemented?

Discussion

(a) The following steps would be simplified. In fact, they would simply be removed, since their purpose is to create a `caresFor` link between `patient8` and `team2`.

4a `patient8` records a reference to `team2`.

4b `team2` records a reference to `patient8`.

(b) None of the steps would be made more complicated, because no other step involves `caresFor`.

Exercise 8

Consider the walk-through below for the valid scenario of *Treat Patient*, which we considered in Subsection 3.1. The objects of the scenario are illustrated in Figures 23(a) and (b). For the purposes of this exercise we have augmented the original diagram to depict the `ConsultantDoctor` object involved.

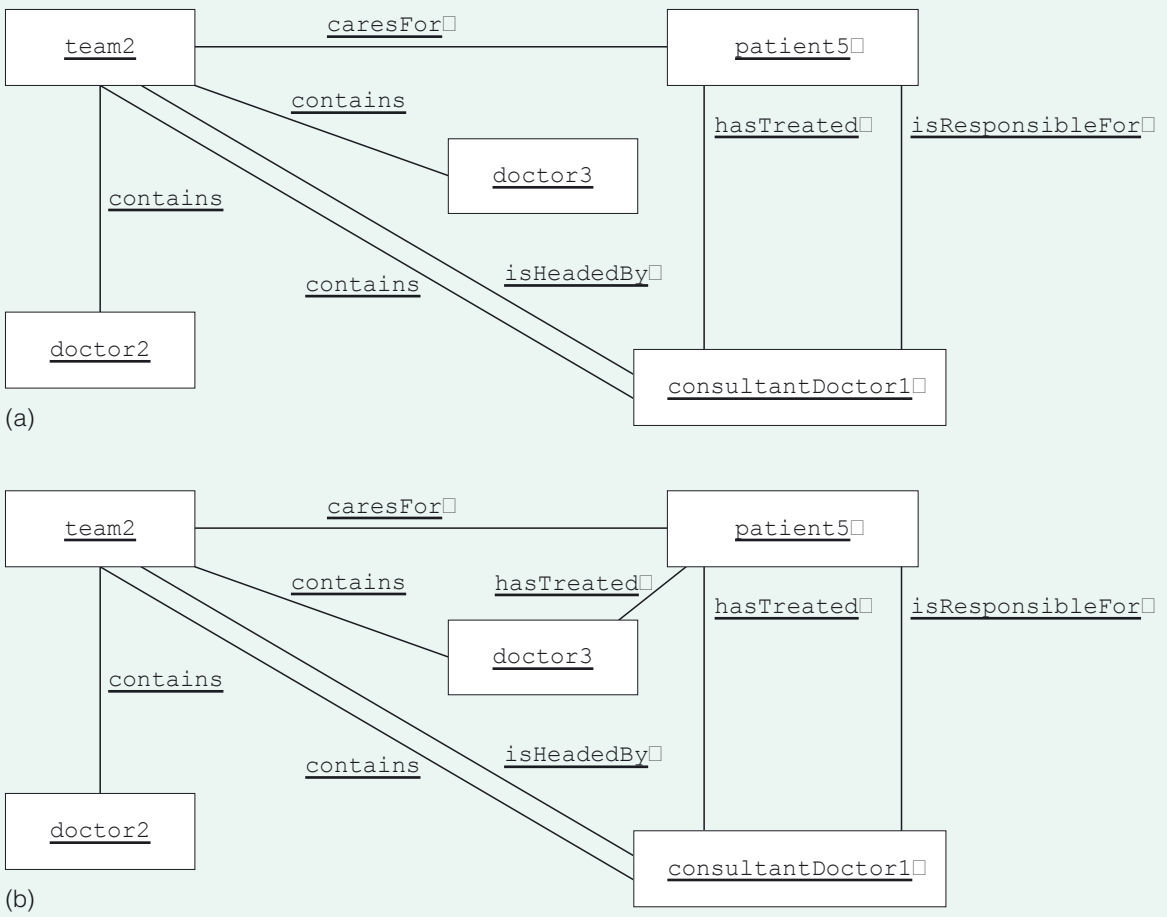

Figure 23 (a) Object diagram for the valid scenario before *Treat Patient*: `patient5` and `doctor3` are not linked. (b) Object diagram for the valid scenario after *Treat Patient*: a `hasTreated` link exists between `patient5` and `doctor3`.

Walk-through:

Given: the `Patient` object `patient5`, and the `Doctor` object `doctor3`.

1 Access the `Team` object linked to `patient5`, which in this case is `team2`.

2 Access the collection of `Doctor` objects linked to `team2`, which in this case is {`doctor1`, `doctor2`, `doctor3`}.

3 Check whether `doctor3` is in {`doctor1`, `doctor2`, `doctor3`}.

As the check succeeds:

4 `patient5` records a reference to `doctor3`.

Describe, in outline, the changes that would be required to this walk-through if `caresFor` were not implemented.

Discussion

Step 1 of the walk-through requires the navigation of `caresFor`, from `Patient` to `Team`, in order to locate the required `Team` object.

If `caresFor` were not implemented, then the `Team` object would have to be accessed by first navigating `isResponsibleFor` from `Patient` to `ConsultantDoctor`, and then navigating `isHeadedBy` from `ConsultantDoctor` to `Team`.

The navigation of `isHeadedBy` from `ConsultantDoctor` to `Team` would be a new navigation, not required by any other use case.

In a situation where two associations (or, indeed, attributes) are derived, each derivation involving the other element, the consequences of not implementing each element must be compared. We will not do that here, but simply state our intention *not* to implement `isResponsibleFor` but to implement `caresFor`. We amend the designs accordingly, and indicate this decision in the implementation model (see Section 5 of the *Handbook*) by enclosing the name of the association `isResponsibleFor` on the class diagram in brackets, as shown in Figure 24.

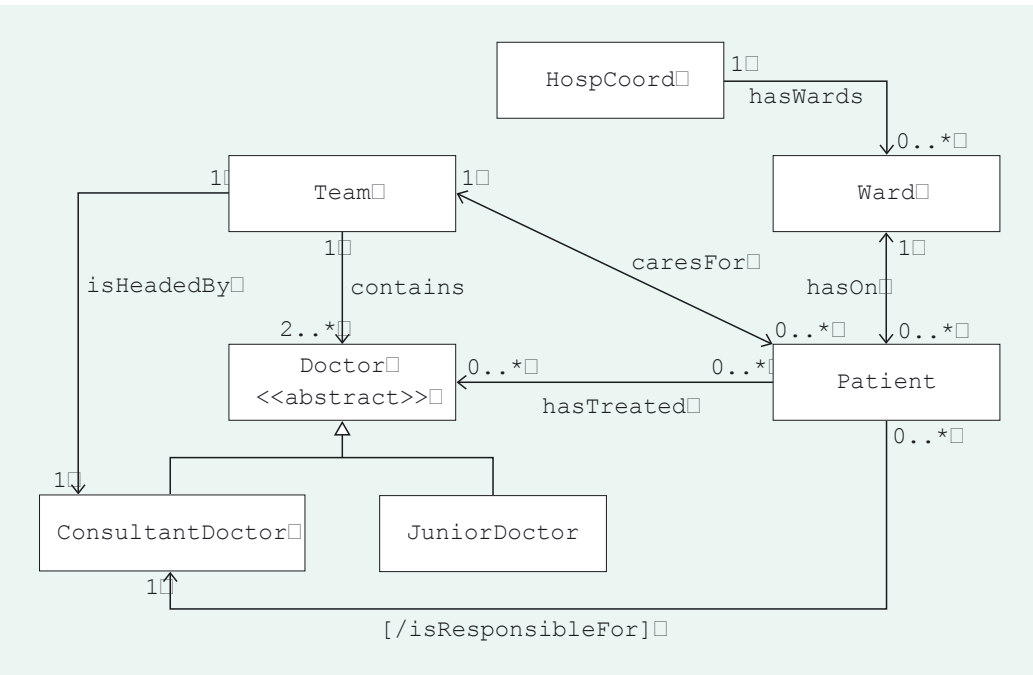

Figure 24 Class diagram for the Hospital System: `isResponsibleFor` will not be implemented

Exercise 9

How should the sequence diagram for *Admit Patient* (Figure 19) be amended now that `isResponsibleFor` is not to be implemented?

Discussion

The parts of the sequence diagram concerned with the message `getConsultantDoctor()`, sent from `patient8` to `team2` (which then enable the creation of an `isResponsible` link between `patient8` and `consultantDoctor2`), should be removed. That is, the lower part of the diagram should be as shown in Figure 25.

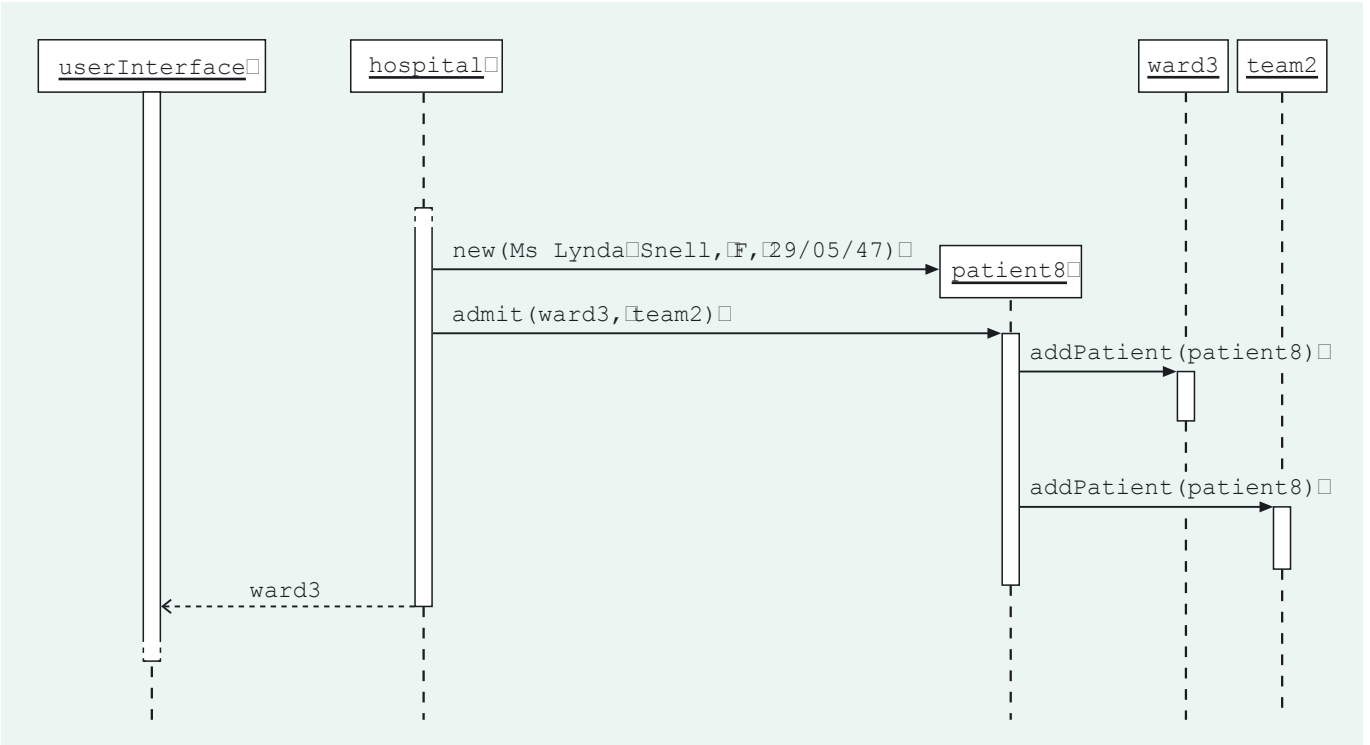

Figure 25 Part of the sequence diagram for *Admit Patient* – without `isResponsibleFor`

In this section you have learnt that derived associations and attributes are often not implemented, and you have seen how to determine and document the impact on existing designs of a decision not to implement such an element. You also saw that there can be a choice of which association or attribute to implement.

5 Communication between user interface and core system

Our investigation of the behaviour of the Hospital System began in *Unit 6* by considering communication between the user interface and the coordinating object. The general pattern of communication is the following.

- The user interface initiates a use case by sending the coordinating object a coordinating message, typically with arguments that are core objects (that is, objects of the core system).
- The coordinating object returns any information that the user interface requires, also in the form of core objects.

So far, we have concentrated on determining what the coordinating object must do when it receives a coordinating message. In this section, we determine what the user interface needs in order to send a coordinating message, and also what it will do with any core objects it receives back as a result. We also decide what accessibility (public, package etc.) should be specified for methods and instance variables of core system classes.

5.1 Objects required as arguments to coordinating messages

First we will consider how the user interface can be supplied with objects so that it can then provide suitable arguments to coordinating messages.

We will take as an example the *List Ward's Patients* use case (labelled E in the requirements document). Here is its description.

> The administrator identifies the ward. For each patient on the ward the system displays the patient's name and age in years.

In *Unit 6*, Subsection 3.1, we specified the coordinating method for this use case as follows.

```
Collection<Patient> getPatients(Ward aWard)
```

> Post-condition: returns a collection of all the `Patient` objects linked to `aWard`.

Figure 26 shows the sequence diagram formed as part of the design for the particular scenario used in *Unit 6*.

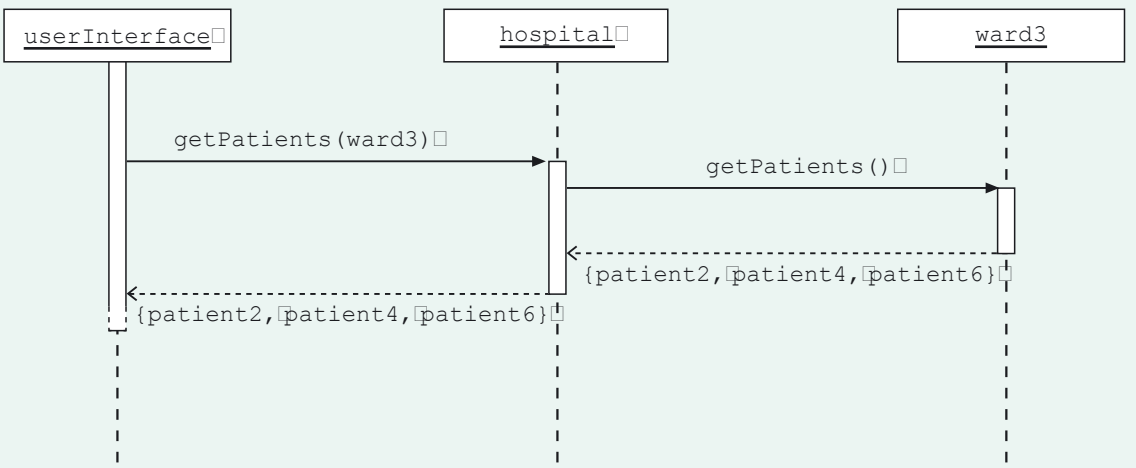

Figure 26 Sequence diagram for *List Ward's Patients*

At this stage we are not concerned with the details of *how* the user interface enables the user to make these selections, but with *what* information it requires.

This sequence diagram shows the user interface sending the coordinating message `getPatients(ward3)`. The argument in this message is a `Ward` object, `ward3`. To be able to send this message, the user interface will first need access to *all* the `Ward` objects in the core system, so that it can supply as argument the `Ward` object corresponding to a ward selected by the user. How will it be able to gain this access?

Recall that, corresponding to the navigation of the association `hasWards` from `HospCoord` to `Ward` (see Figure 24), there is an instance variable `wards` in `HospCoord` that references a collection of all the `Ward` objects. Consequently, it is possible to provide a method `getWards()` in the `HospCoord` class which returns a collection of *all* the `Ward` objects in the core system, and which can then be used by the user interface to retrieve these `Ward` objects. We therefore add the following method specification to the `HospCoord` class description.

`Collection<Ward> getWards()`

Post-condition: returns a collection of all the `Ward` objects.

The sequence diagram in Figure 27 below illustrates what happens when the system is in action. It shows the user interface requesting all the `Ward` objects ({`ward1`, `ward2`, `ward3`, ...}) by sending a `getWards()` message to `hospital`, and then initiating the use case. The lower part of this sequence diagram, showing the interactions involved when the coordinating method is invoked, is exactly the same as in Figure 26.

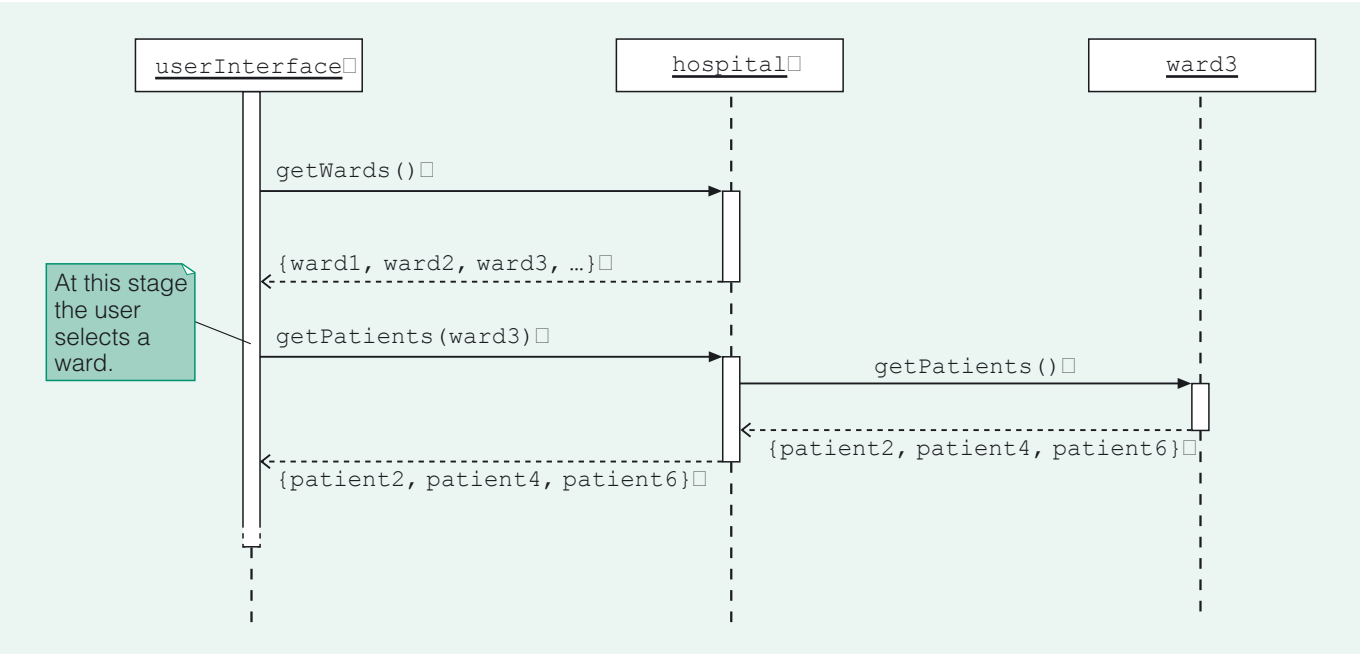

Figure 27 Sequence diagram illustrating `getWards()`

Exercise 10

The *List Team's Patients* use case (labelled F in the requirements document) is described as follows.

> The administrator identifies the team. For each patient cared for by the team, the system displays:
>
> - the patient's name;
> - the name of the ward that the patient is on.

From this description we formed the following coordinating method specification in *Unit 6*, Subsection 3.2:

`Map<Patient, Ward> getPatientsAndWards(Team aTeam)`

> Post-condition: returns a map of pairs of `Patient` and `Ward` objects, such that for each `Patient` object `aPatient` linked to `aTeam`, the map contains the key–value pair (`aPatient`, `aWard`), where `aWard` is linked to `aPatient`.

(a) Describe a method with which `HospCoord` should be equipped to enable the user interface to provide an appropriate argument for this coordinating method.

(b) What additional association would be needed in order to provide the method identified in part (a), and in which direction does this association need to be navigated? Which class has a corresponding responsibility for holding link references, and how should this responsibility be documented?

Discussion

(a) `HospCoord` requires a method that returns a collection of all the `Team` objects in the system. We could call this method `getTeams()`.

(b) The coordinating object, `hospital`, will need to access all the `Team` objects in the system. This means that we need an association between `HospCoord` and `Team` navigated from `HospCoord` to `Team`. We will call this association `hasTeams`. The following responsibility for holding link references is needed in `HospCoord`.

> `Collection<Team> teams`
>
> References a collection of all `Team` objects.

For many use cases, supplying the user interface with the core objects it requires so that it can initiate a use case is straightforward, as the above examples illustrate. Often, this means supplying *all* the core objects of a particular class – all `Team` objects, for example – so that the user interface can allow the user freedom of selection – any team, for example.

But when a coordinating method has a pre-condition, it can be sensible to provide restricted collections of core objects to help the user interface ensure that the pre-condition is met. To illustrate this, we will consider the *Treat Patient* use case (labelled B in the requirements document). Here is the specification of its coordinating method and part of the design we created in Subsection 3.1 of this unit.

Coordinating method specification:

`void recordTreatment(Patient aPatient, Doctor aDoctor)`

Pre-condition: `aDoctor` and `aPatient` must be linked to the same `Team` object; if not, an exception is thrown.

Post-condition: `aDoctor` is linked to `aPatient`.

Walk-through for the valid scenario:

Given: the `Patient` object `patient5`, and the `Doctor` object `doctor3`.

1 Access the `Team` object linked to `patient5`, which in this case is `team2`.

2 Access the collection of `Doctor` objects linked to `team2`, which in this case is {`doctor1`, `doctor2`, `doctor3`}.

3 Check whether `doctor3` is in {`doctor1`, `doctor2`, `doctor3`}.

As the check succeeds:

4 `patient5` records a reference to `doctor3`.

Figure 28 shows the corresponding sequence diagram.

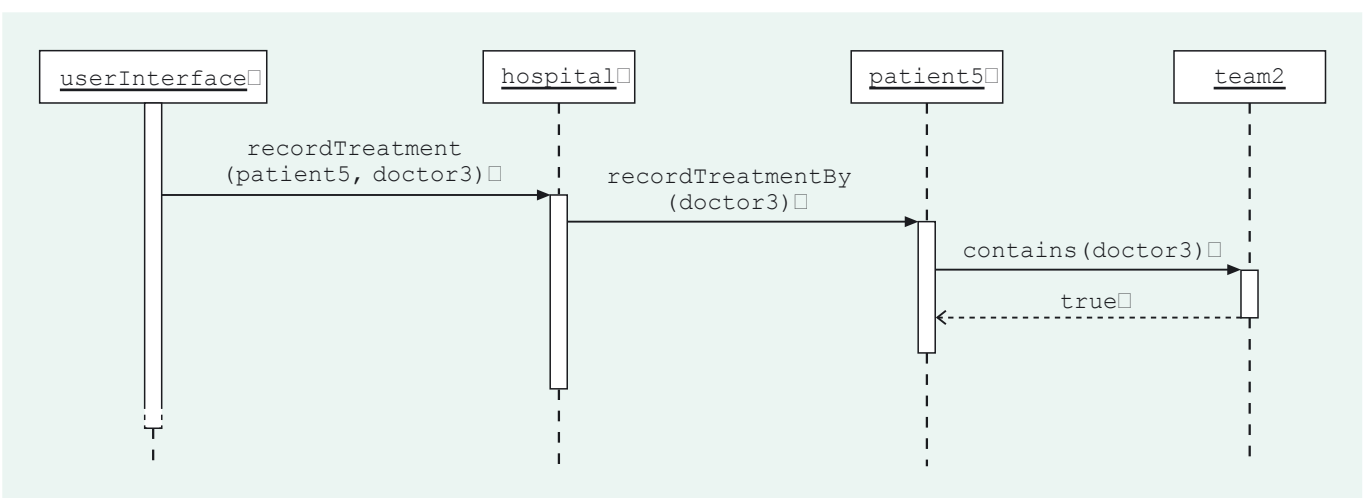

Figure 28 Sequence diagram for *Treat Patient* (valid scenario)

To initiate the use case, the user interface must send a message with two arguments: a `Patient` object and a `Doctor` object. To enable it to do this, one possibility is for the user interface to be provided with *all* the `Patient` objects and *all* the `Doctor` objects, to allow the user free choice of any patient and any doctor.

But there is a pre-condition on the coordinating method, reflecting an invariant (number 7):

If `aPatient` is any `Patient` object, then any `Doctor` object linked to `aPatient` is also linked to the `Team` object which is linked to `aPatient`.

Within our design for the coordinating method we have incorporated a check that this invariant is not about to be broken. Without such a check, if the pre-condition has not been met, then the system could be put into an invalid state. This is, in a sense, the core system's second line of defence because it would be preferable for the user interface to adhere to the pre-condition in the first place. How can the user interface do this? One way is to allow the user to select the patient whose treatment is to be recorded, and for the user interface then to list only those doctors on that patient's team, so that the user can select an appropriate doctor. If the user interface is able to determine the team that cares for a patient and then all the doctors that are in a particular team, it is equipped to ensure that the pre-condition is met before sending the coordinating message.

So the following methods are needed in `HospCoord`.

- `getPatients()` The user interface will use this to get all the `Patient` objects, thus enabling the user to select any patient. This is a relatively straightforward method that returns all the `Patient` objects (navigating the associations `hasWards` and `hasOn`), and we will not consider it further here.
- `getTeam(aPatient)` The user interface will use this to find the team that cares for a particular patient. Fortunately, there is already such a method in `HospCoord` (introduced as a coordinating method for the *List Patient's Treatment* use case in *Unit 6*, Subsection 3.3). This same method can be used here.
- `getDoctors(aTeam)` The user interface will use this to find the doctors in a particular team, thus restricting the user's selection to a doctor in the right team.

This last method, `getDoctors(aTeam)`, merits a little consideration. To determine a design for it, we can take the same approach as for coordinating methods: consider what is required in the core system for a typical scenario, and what interaction between objects will achieve what is required.

Consider a scenario where all the `Doctor` objects in team `team2` are wanted. Suppose that there are three doctors in the team, corresponding to `doctor1`, `doctor2` and `doctor3` as illustrated in Figure 29.

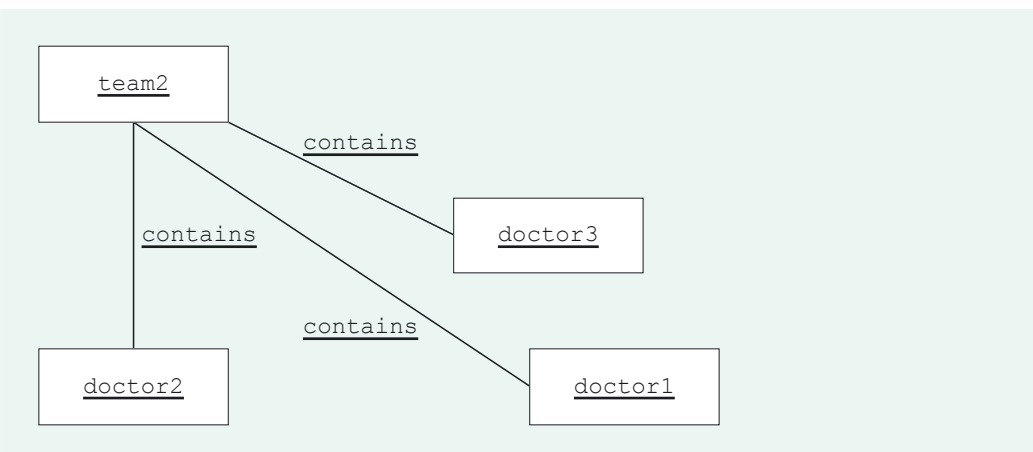

Figure 29 `team2` contains `doctor1`, `doctor2` and `doctor3`

The method can be specified as follows.

```
Collection<Doctor> getDoctors(Team aTeam)
```

Post-condition: returns a collection of all the `Doctor` objects linked to `aTeam`.

A walk-through for the scenario is:

Given: the `Team` object `team2`.

1 Access the collection of `Doctor` objects linked to `team2`, which in this case is {`doctor1`, `doctor2`, `doctor3`}.

2 Return {`doctor1`, `doctor2`, `doctor3`}.

A corresponding sequence diagram is shown in Figure 30.

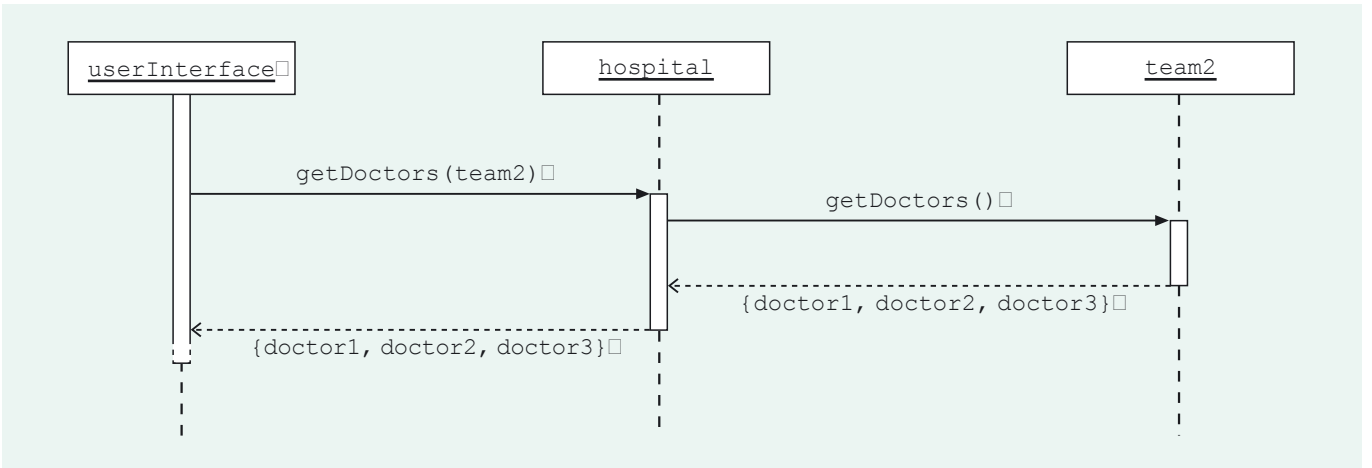

Figure 30 Sequence diagram for `getDoctors(aTeam)`

Exercise 11

What are the consequences of the above design for the class descriptions in the structural part of the implementation model?

Discussion

The consequences for the class descriptions are:

1 In `HospCoord` we must add the following:

Protocol

`Collection<Doctor> getDoctors(Team aTeam)`

Post-condition: returns a collection of all the `Doctor` objects linked to `aTeam`.

2 In `Team` we must add the following:

Links

`Collection<Doctor> doctors`

References a collection of all the linked `Doctor` objects.

Protocol

`Collection<Doctor> getDoctors()`

Post-condition: returns a collection of all the linked `Doctor` objects.

The user interface is now equipped to ensure the pre-condition for the coordinating method `recordTreatment(aPatient, aDoctor)`.

It will be helpful to review how this all works. The single sequence diagram in Figure 31 showing all the message passing involved for this scenario looks quite complex, but each interaction follows logically from the previous one, as we describe in the discussion following the figure.

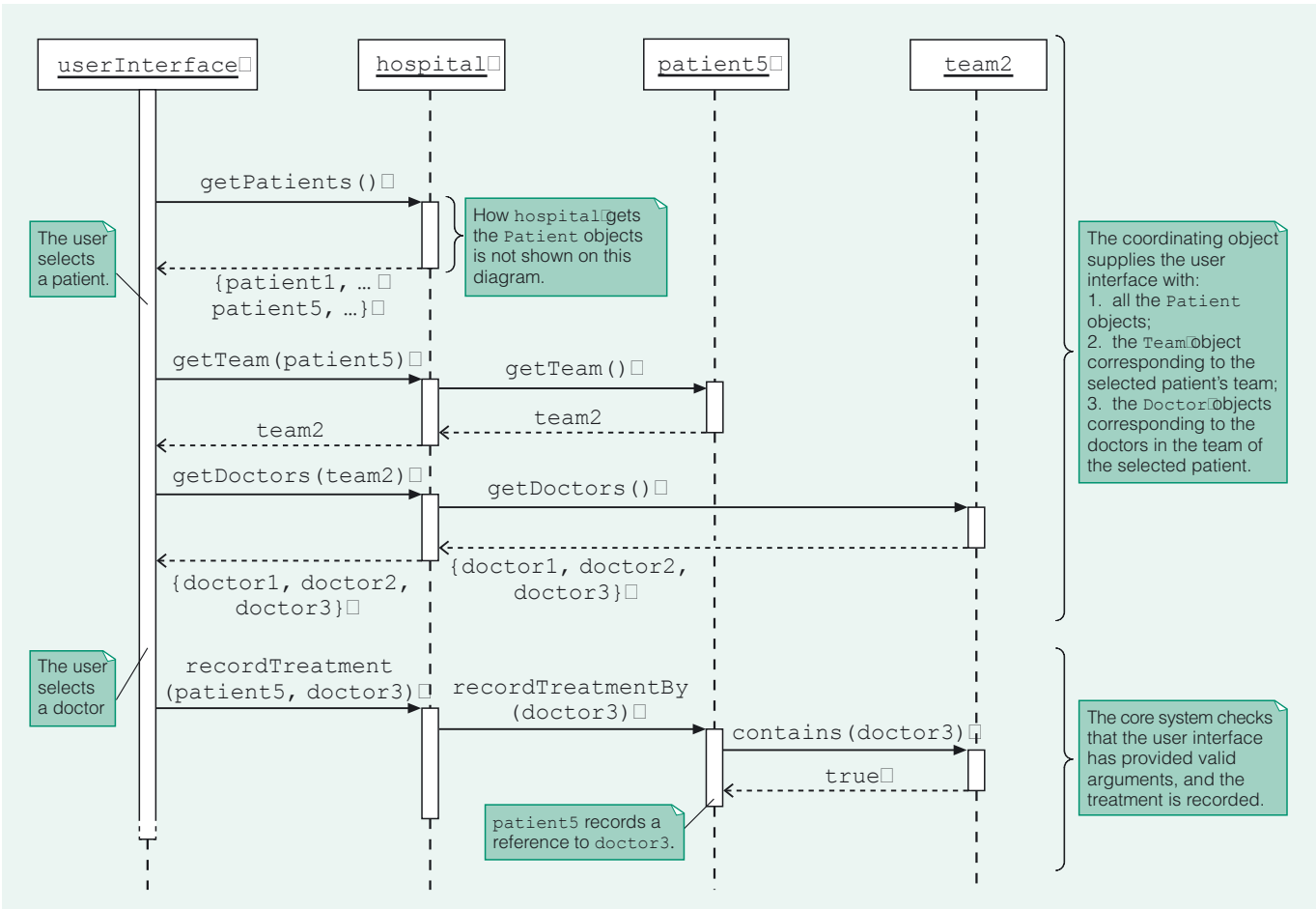

Figure 31 Complete sequence diagram for *Treat Patient*

Analysing this figure, we see that:

1 First the user interface sends a `getPatients()` message to the coordinating object to get all the `Patient` objects.

2 Having identified the `Patient` object, `patient5`, corresponding to the user's selection, the user interface then sends the coordinating object the message `getTeam(patient5)` to find the `Team` object corresponding to the team that cares for the patient.

3 When `team2` is returned, the user interface sends the message `getDoctors(team2)` to find the `Doctor` objects corresponding to the doctors that are in the team and hence allowed to treat the patient.

4 The user interface can then provide a restricted list of doctors to the user. The user selects an appropriate doctor, here corresponding to `doctor3`.

5 The user interface sends a message `recordTreatment(patient5, doctor3)` whose arguments satisfy the pre-condition.

6 The core system checks that the user interface has provided valid arguments, and the treatment is recorded.

Remember that we are not concerned with exactly how the user interface identifies the `Patient` object or the `Doctor` object corresponding to the user's selection.

SAQ 14

When a coordinating method such as `recordTreatment(aPatient, aDoctor)` has a pre-condition that should ensure an invariant will not be broken, why is it sensible for steps to be taken within the method to preserve the invariant?

ANSWER

It is possible that faulty code, in the user interface for example, might cause the method to be invoked when the pre-condition does not hold. By checking that the invariant is not about to be broken, the integrity of the core system is maintained.

5.2 Retrieving attribute values

Again, we are not concerned here with precisely *how* the user interface displays the information, but simply with *what* information is involved.

We have discussed what the user interface needs in order to send coordinating messages with valid arguments. The user interface also needs to be able to *display* certain attribute values of the objects returned to it. To allow the user interface to do this simply requires getter methods to be provided for the required attributes.

Consider *List Ward's Patients.* The requirements state that the name and age of each patient are displayed. The coordinating method, `getPatients(aWard)`, returns a collection of `Patient` objects. If the user interface can send messages `getName()` and `getAge()` to `Patient` objects, it can get the information it needs for its display. We therefore add the following specifications to the `Patient` protocol.

In *Unit 6*, Subsection 5.2, we said that, in specifying methods during design, if it is not obvious what type is appropriate then one is 'invented' and specified fully later when more is known about how it is used. Here we assume a `Name` type for a patient's name, and an integer `(int)` for their age.

`Name getName()`

Post-condition: returns `name`.

`int getAge()`

Post-condition: returns `age`.

SAQ 15

Recall our decision not to implement `age`, a derived attribute, as an instance variable. How will the method `getAge()` be implemented?

ANSWER

The implementation of this method will use the derivation of `age`, which is based on the following invariant (number 2).

> For each `Patient` object, the value of its `age` attribute is equal to the difference (in years) between the current date and the value of its `dateOfBirth` attribute.

It would be sufficient to have getter methods only for those attributes whose values are explicitly mentioned in the requirements. But to allow for the possibility of future user interfaces having different display requirements (for example, displaying a patient's date of birth rather than their age), it is sensible to include a getter method for *each* attribute. For `Patient`, for example, a getter method will be specified for each of `name`, `sex`, `dateOfBirth` and `age` in our implementation model.

5.3 Accessibility

Throughout the course we have referred to communication between the user interface and core objects as being mediated by the coordinating object, which acts as a gatekeeper to the core system. The user interface sends messages to the coordinating object, which initiates appropriate interactions within the core system.

However, in Subsection 5.2 we suggested that the user interface be allowed to communicate directly with core objects, by sending them getter messages.

In general, allowing the user interface to send messages directly to core objects could have serious consequences for the integrity of the core system. For example, if the user interface has access to a `Patient` object and a `Ward` object, what is there to stop it sending an `addPatient(aPatient)` message to the `Ward` object, and adding a patient to a ward that it should not be on (perhaps the ward is of the wrong type)? Clearly it would not be sensible to do this, and the developer of the user interface should not write this kind of inappropriate code. However, it would be preferable to prevent the user interface from sending messages that invoke methods such as `addPatient(aPatient)`, which are designed to be used only within the core system.

Java and certain other languages provide for this kind of arrangement. In *Unit 5*, Section 5, you saw how Java packages and access modifiers could be used to prevent inappropriate use of methods and instance variables by objects in different parts of a system. We started our design process in *Unit 6* with this in mind, stating that the user interface and the core system would be designed as separate components (packages, in Java).

We can now make further use of these ideas to specify appropriate accessibility of the attributes and links (implemented by instance variables), and methods specified for the classes of the core system. Three kinds of accessibility – public, private and package – are relevant.

SAQ 16

Briefly explain the meaning of each of the following in Java, and how each is achieved in a Java program:

(a) public accessibility;

(b) private accessibility;

(c) package accessibility.

ANSWER.....

(a) Any class in any package can access a member having public accessibility. This type of accessibility is achieved by prefacing a class member (e.g. an instance variable or method) with the access modifier `public`.

(b) Only a class itself has access to one of its members having private accessibility. This type of accessibility is achieved by prefacing a class member with the access modifier `private`.

(c) All classes in the same package have access to a member having package accessibility. This type of accessibility is achieved by *not* prefacing a class member with an access modifier.

Public accessibility

Any core system method designed to be invoked by the user interface should have public accessibility, because the user interface and the core system are in separate packages. There are three kinds of method in this category. We will use examples from the *List Ward's Patients* use case (labelled E in the requirements document) discussed in Subsection 5.1 above to illustrate them.

1 Use case coordinating methods of the coordinating class, for example the method `getPatients(aWard)` of `HospCoord`. A coordinating method exists to allow the user interface to initiate a use case by sending a coordinating message.

2 Methods of the coordinating class designed to provide the user interface with objects to use as coordinating message arguments. For example, the method `getWards()` of `HospCoord` enables the user interface to send, as the argument to the *List Ward's Patients* coordinating method, the `Ward` object corresponding to the ward selected by the user.

3 Getter methods specified for core classes to enable the user interface to display information. For example, the method `getAge()` of `Patient` is required for the user interface to display a patient's age.

Package accessibility

Package accessibility should be specified for core system methods that the user interface should not invoke, but which should be invoked by other core objects. The methods designed to fulfil the specification of a coordinating method should typically only be invoked within the core system. For example, the method `getPatients()` of `Ward` is invoked only by `HospCoord`, as part of its coordination role for *List Ward's Patients*. The method should therefore be specified as having package accessibility.

Private accessibility

Private accessibility is appropriate when a method is only invoked by the same class of objects. We have no such examples within the Hospital System. However, in accordance with common conventions (and as discussed in *Unit 5*), the attributes and link references specified for a class should have private accessibility, because they will be implemented as instance variables.

Specifying access levels

In UML method specifications, to keep designs general, the names of public elements are prefixed with a + sign, and the names of private elements are prefixed with a – sign. However, since our target language is Java, which uses the modifiers `public`, `private`, etc., for simplicity we will simply preface elements in the design with these modifiers. If there is no prefaced modifier, package accessibility is assumed.

For example, the specification of the coordinating method `getPatients(aWard)` of `HospCoord` becomes:

```
public Collection<Patient> getPatients(Ward aWard)
```

Post-condition: returns a collection of all the `Patient` objects linked to `aWard`.

Exercise 12

The following is now part of the description of the `Team` class.

Attributes

`private String code` The unique code of the team

Links

`private Collection<Patient> patients`

References a collection of all the linked `Patient` objects.

What accessibility do `code` and `patients` have, and why?

Discussion.....

They both have private accessibility. This is because they will both be implemented as instance variables.

Different access levels for similar methods

Our designs specify, amongst other methods, the following two methods for the `Ward` class.

`int getCapacity()`

Post-condition: returns `capacity`.

`Collection<Patient> getPatients()`

Post-condition: returns a collection of all the linked `Patient` objects.

SAQ 17

Explain what accessibility should be specified for each of the above methods, and how this specification should be documented.

ANSWER

The method `getCapacity()` is a getter method, whose purpose is to enable the user interface to display information. It should have public accessibility, with the method specification being prefaced by `public`.

The method `getPatients()`, as we discussed above, is intended to be invoked by the coordinating object as part of its coordinating role for *List Ward's Patients*. It should have package accessibility, with the method specification having no prefaced modifier.

In fact, the methods `getCapacity()` and `getPatients()` will be implemented in similar ways – `getCapacity()` by returning the `capacity` instance variable of the receiver `Ward` object, and `getPatients()` by returning its `patients` instance variable. It is important to understand why `getCapacity()` has public accessibility whilst `getPatients()` has package accessibility, despite their apparent similarities. The answer lies in the fundamental difference in the instance variables in question.

The instance variable `capacity` implements a simple attribute whose values are integers. There is no potential for harm in the user interface's invoking a method such as `getCapacity()` (so long as care is taken that this does not lead to a privacy leak, as discussed in *Unit 5*, Section 6).

The instance variable `patients` on the other hand implements a set of links between a `Ward` object and its `Patient` objects. It references a collection of core objects. To protect the independence of the user interface and the core system, the user interface should only be able to discover information related to the structure of the system (such as which objects are linked) by asking the coordinating object. It should not be able to send a `getPatients()` message (which involves the links between `Ward` and `Patient` objects) directly to a `Ward` object.

In this section, you have learnt that additional methods must be specified for core system classes so that the user interface has the information it needs to initiate a use case, and the means to display information about any results returned. You have also learnt how to determine and document the accessibility of attributes, link references and methods of the core system classes.

6 Summary

In this unit the design process progressed beyond the single design for each use case that was presented in *Unit 6*. You considered different designs for use cases, and learnt about some design principles that can help when assessing the relative merits of different designs. You saw how designs can deal with alternative scenarios, and you met ways of ensuring that invariants are not broken. You were introduced to what can be involved in deciding whether or not to implement a derived attribute or association. Finally, you learnt more about how to design for appropriate communication between the user interface and the core system.

Throughout this unit, we used examples from the Hospital System as illustrations. The decisions taken here required amendments and additions to the designs already created for this system in *Unit 6*, and consequently led to changes to the implementation model which is being formed throughout the design process. You examined some of these changes, for example, the addition and amending of method specifications, and the addition of an association.

All the design information for the Hospital System that was created in this unit has been gathered together in the Appendix. It represents the structural model for the Hospital System corresponding to its current stage of development, and we will build on it in subsequent units.

The next unit, *Unit 8*, gives you practice in using the design techniques and ideas introduced so far. In *Unit 9* you will learn about the detailed design necessary to complete the implementation model for the Hospital System. Among the kinds of decision still to be taken are the following.

The complete implementation model for the Hospital System is in Section 5 of the *Handbook*.

- Should any other classes be introduced, for example to give an appropriate class hierarchy?
- For attributes such as patients' name and sex, we have so far used 'undefined' types such as `Name` and `Sex`: we need to decide exactly what these types should be. Should names just be strings, or something more complex with separate first name and surname, for example? Some of these types may require new classes to be defined.

LEARNING OUTCOMES

After studying this unit, you should be able to:

- describe and use each of this unit's key terms (summarised in the Glossary);
- compare different designs by considering potential implementations;
- make appropriate use of design principles;
- explain the difference between an alternative scenario and an invalid scenario;
- create designs to cater for invalid and alternative scenarios;
- explain why the core system may need to check that an invariant is not going to be broken;
- identify the consequences for a design if a derived attribute or association is not implemented;
- specify methods required to give the user interface access to the objects it requires to initiate a use case;

- specify methods required to enable the user interface to access the attribute values of core objects;
- specify appropriate access levels for core system attributes, link references and methods;
- identify the consequences of design decisions for the structural part of the implementation model.

Appendix

Hospital System implementation model (the structure so far)

For reference, this Appendix contains the structural model of the system corresponding to the point we have reached in the design process at the end of *Unit 7*.

Class diagram

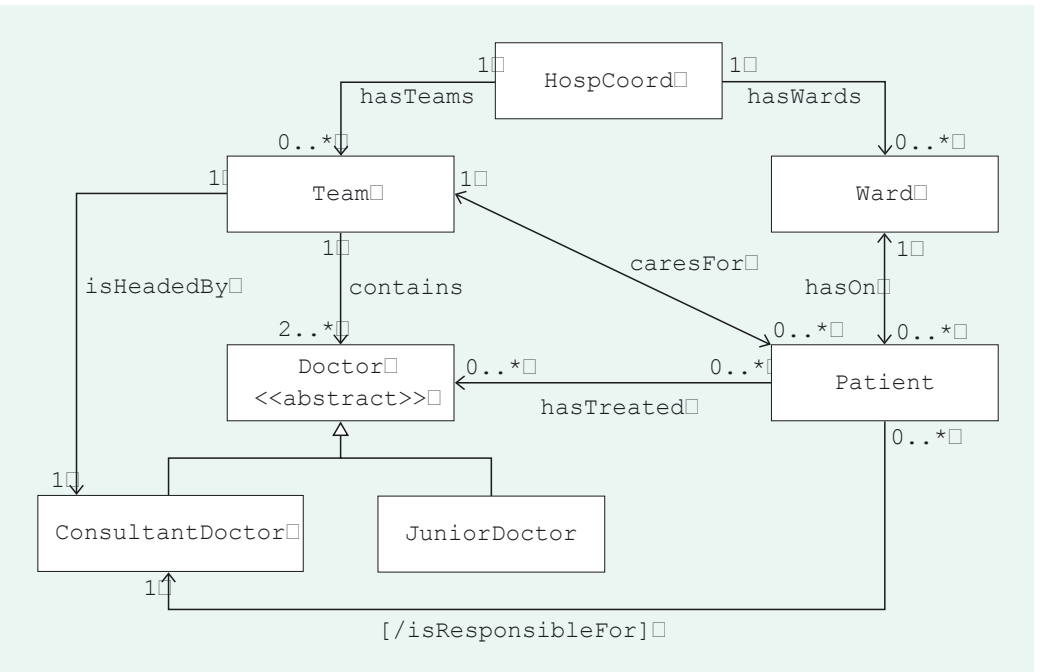

Figure 32 Class diagram for the Hospital System

Class descriptions

Class `HospCoord` The coordinating class

Attributes

None

Links

`private Collection<Ward> wards`
References a collection of all `Ward` objects.

`private Collection<Team> teams`
References a collection of all `Team` objects.

Protocol

`public Collection<Patient> getPatients(Ward aWard)`
Post-condition: returns a collection of all the `Patient` objects linked to `aWard`.

`public Map<Patient, Ward> getPatientsAndWards(Team aTeam)`

Post-condition: returns a map of pairs of `Patient` and `Ward` objects, such that for each `Patient` object `aPatient` linked to `aTeam`, the map contains the key–value pair (`aPatient`, `aWard`), where `aWard` is linked to `aPatient`.

`public ConsultantDoctor getConsultantDoctor(Patient aPatient)`

Post-condition: returns the `ConsultantDoctor` object linked to `aPatient`.

`public Team getTeam(Patient aPatient)`

Post-condition: returns the `Team` object linked to `aPatient`.

`public Collection<Doctor> getDoctors(Patient aPatient)`

Post-condition: returns a collection of all the `Doctor` objects linked to `aPatient`.

`public void recordTreatment(Patient aPatient, Doctor aDoctor)`

Pre-condition: `aDoctor` and `aPatient` must be linked to the same `Team` object; if not, an exception is thrown.

Post-condition: `aDoctor` is linked to `aPatient`.

`public Ward admit(Name aName, Sex aSex, M256Date aDate, Team aTeam)`

Post-condition: if there is no `Ward` object of the appropriate type with at least one free bed then `null` is returned.

Otherwise a new `Patient` object, `aPatient`, is created with the supplied attribute values, and `age` according to `aDate`, and

(i) `aPatient` is linked to `aWard`, where `aWard` is a `Ward` object of the appropriate type with the greatest number of free beds, and `numberOfFreeBeds` of `aWard` is decremented;

(ii) `aPatient` is linked to `aTeam`;

(iii) `aPatient` is linked to the `ConsultantDoctor` object that is linked to `aTeam`;

(iv) `aWard` is returned.

`public Collection<Ward> getWards()`

Post-condition: returns a collection of all the `Ward` objects.

`public Collection<Team> getTeams()`

Post-condition: returns a collection of all the `Team` objects.

`public Collection<Patient> getPatients()`

Post-condition: returns a collection of all the `Patient` objects.

`public Collection<Doctor> getDoctors(Team aTeam)`

Post-condition: returns a collection of all the `Doctor` objects linked to `aTeam`.

Class `Ward` — A hospital ward

Attributes

`private String name`	The unique name of the ward
`private Sex type`	Whether the ward is for male or female (M or F) patients
`private int capacity`	The maximum number of patients that can be on the ward at any one time
[`int /numberOfFreeBeds`	The number of free beds on the ward]

Links

`private Collection<Patient> patients`

References a collection of all the linked `Patient` objects.

Protocol

`public String getName()`

Post-condition: returns `name`.

`public Sex getType()`

Post-condition: returns `type`.

`public int getCapacity()`

Post-condition: returns `capacity`.

`public int getNumberOfFreeBeds()`

Post-condition: returns `numberOfFreeBeds`.

`Collection<Patient> getPatients()`

Post-condition: returns a collection of all the linked `Patient` objects.

`void addPatient(Patient aPatient)`

Post-condition: a reference to `aPatient` is recorded; `numberOfFreeBeds` is decremented.

Class `Team` — A team of doctors

Attributes

`private String code` — The unique code of the team

Links

`private Collection<Patient> patients`

References a collection of all the linked `Patient` objects.

`private ConsultantDoctor consultantDoctor`

References the linked `ConsultantDoctor` object.

`private Collection<Doctor> doctors`

References a collection of all the linked `Doctor` objects.

Protocol

`public String getCode()`

Post-condition: returns `code`.

`Map<Patient, Ward> getPatientsAndWards()`

Post-condition: returns a map of pairs of `Patient` and `Ward` objects, such that for each `Patient` object, `aPatient`, linked to the receiver, the map contains the key–value pair (`aPatient`, `aWard`) where `aWard` is linked to `aPatient`.

`void addPatient(Patient aPatient)`

Post-condition: a reference to aPatient is recorded.

`ConsultantDoctor getConsultantDoctor()`

Post-condition: returns the linked `ConsultantDoctor` object.

`boolean contains(Doctor aDoctor)`

Post-condition: returns `true` if `aDoctor` is linked to the receiver, `false` otherwise.

`Collection<Doctor> getDoctors()`

Post-condition: returns a collection of all the linked `Doctor` objects.

Class `Patient` — A patient in the hospital

Attributes

`private Name name`	The name of the patient
`private Sex /sex`	The sex (M or F) of the patient
`private M256Date dateOfBirth`	The date of birth of the patient
`[int /age`	The age of the patient in years]

Links

`private Ward ward`

References the linked `Ward` object.

`private Team team`

References the linked `Team` object.

`private Collection<Doctors> doctors`

References a collection of all the linked `Doctor` objects.

`[ConsultantDoctor consultantDoctor`

References the linked `ConsultantDoctor` object.]

Constructor

`Patient(Name aName, Sex aSex, M256Date aDate)`

Post-condition: initialises a new `Patient` object with the given attribute values and `age` according to `aDate`.

Protocol

`public Name getName()`

Post-condition: returns `name`.

`public Sex getSex()`

Post-condition: returns `sex`.

`public int getAge()`

Post-condition: returns `age`.

`public M256Date getDateOfBirth()`

Post-condition: returns `dateOfBirth`.

`public String toString()`

Post-condition: returns a string representation of `name`, `sex` and `dateOfBirth`.

`Ward getWard()`

Post-condition: returns the linked `Ward` object.

`Team getTeam()`

Post-condition: returns the linked `Team` object.

`Collection<Doctor> getDoctors()`

Post-condition: returns a collection of all the linked `Doctor` objects.

`ConsultantDoctor getConsultantDoctor()`

Post-condition: returns the linked `ConsultantDoctor` object.

`void recordTreatmentBy(Doctor aDoctor)`

Pre-condition: `aDoctor` and the receiver must be linked to the same `Team` object; if not, an exception is thrown.

Post-condition: a reference to `aDoctor` is recorded.

`void admit(Ward aWard, Team aTeam)`

Post-condition: a reference to `aWard` is recorded, `aWard` records a reference to the receiver, and `numberOfFreeBeds` of `aWard` is decremented. A reference to `aTeam` is recorded, and `aTeam` records a reference to the receiver. A reference to the `ConsultantDoctor` object linked to `aTeam` is recorded.

Class `Doctor` — A doctor at the hospital
Abstract: generalises `JuniorDoctor` and `ConsultantDoctor`

Attributes

`private Name name`	The name of the doctor

Protocol

`public Name getName()`

Post-condition: returns `name`.

Class `JuniorDoctor` — A junior doctor at the hospital
Specialises `Doctor`

Attributes

`private Grade grade`	The grade (1, 2 or 3) of the junior doctor

Protocol

`public Grade getGrade()`

Post-condition: returns `grade`.

Class `ConsultantDoctor` — A consultant doctor at the hospital
Specialises `Doctor`

Attributes

None

Protocol

None

Glossary

alternative scenario A (valid) scenario for which the use case requirements specify an alternative course of action.

cascading A design where, to get an object `r` to perform a task, an object `p` sends a message to an object `q` asking it to ask `r` to perform its task, and `q` sends a message to `r` asking it to perform its task.

design principle A guideline for which objects should be allocated which kinds of task, and how the objects should communicate.

forking A design where, to get an object `r` to perform a task, an object `p` sends a message to an object `q` asking it for `r`; `q` returns `r` to `p`, and `p` then sends a message to `r` asking it to perform the task.

information expert An object which most readily has to hand the information needed to carry out a particular task.

invalid scenario A scenario where a pre-condition is broken.

valid scenario A scenario where no pre-condition is broken.

Index